Special Topic in Computer Science (CSC649)

Big Data and Machine Learning Project

Predicting Students' Success or Failure in Calculus

3 February 2025

Lecturer Name

KHAIRUL NIZAM BIN ABD HALIM

Group Members

Name	Matric No	Signature
MOHAMMAD SHAHZWAN BIN AGUS ARYADI	2022470776	
MUHAMMAD AQIMAN BIN MOHD FAUZI	2022675706	
MUHAMMAD AKMAL SYAHMI BIN ABD RAZAK	2022646294	
MOHD IMTIYAZ BIN KHAIRUDDIN	2022484576	
MUHAMAD NORSHAFIZAL BIN SAAET	2022484276	

ABSTRACT

The field of calculus education, fundamental to STEM disciplines, faces persistent challenges as many students struggle to master its concepts. This study explores the intricate factors influencing students' success or failure in calculus, leveraging a dataset of 500 respondents. Using the K-Nearest Neighbors (KNN) algorithm, a well-established machine learning approach, the research analyzes student performance based on variables such as prior academic achievements, study habits, and external influences. The predictive model achieves an exceptional 100% accuracy during training and a reliable 98% testing accuracy, validated through k-fold cross-validation. These results provide deep insights into the contributors to calculus performance. It offers a valuable guidance for educators and institutions to implement targeted interventions and improve learning outcomes in mathematics education.

TABLE OF CONTENTS

ABSTRACT **I**

TABLE OF CONTENTS **II**

LIST OF FIGURES **V**

LIST OF TABLES **VIII**

LIST OF ABBREVIATIONS **IX**

1.0 INTRODUCTION **1**

2.0 PROBLEM STATEMENT **1**

3.0 PROJECT OBJECTIVES **1**

4.0 PROJECT SCOPE **2**

5.0 SIGNIFICANCE OF PROJECT **2**

6.0 LITERATURE REVIEW **3**

7.0 DATA COLLECTION **4**

7.1 Data Analytic Purposes 5
7.1.1 Categorical Data 5
7.1.2 Numerical Data 6

7.2 Data Science Purposes 6
7.2.1 Label 6
7.2.2 Features 7

7.3 Target Respondent 8
7.3.1 Data Collection Instrument 8
7.3.2 Distribution Strategy 9

8.0 DATA SCIENCE **9**

8.1 Data Preprocessing 9
8.1.1 Data Cleaning 9
8.1.2 Data Selection 13

8.1.3 Data Transformation 15

8.2 Classification Model 17
8.2.1 Data Split Approach 17
8.2.2 Modeling Classification Model 18
8.2.2.1 Machine Learning Algorithm 19
8.2.2.2 Testing and Performance Analysis 20
8.2.3 Cross-validation Approach (K-Fold) 22
8.2.3.1 List of K-Fold Experiments Batch 25
8.2.3.2 Proposed the Best K-Fold Experiment Batch (The best hyperparameter combination) 29
8.2.4 Remodeling Classification Model (Applying K-Fold Result) 30
8.2.4.1 Testing and Performance Analysis 31

8.3 Product (Prototype of application for prediction project) 32
8.3.1 Applying the Classification Model in Application 32
8.3.2 Run Application Testing 33
8.3.2.1 List of Data Input 33
8.3.2.2 List of Observation Results 35
8.3.2.3 Analysis and Discussion 35

9.0 DATA ANALYSIS AND VISUALIZATION 36

9.1 Categorical Data Analysis 36
9.1.1 Internet Interruption vs. Calculus Result for On-Campus and Off-Campus Students 37
9.1.2 Preparation Methods and Perceived Mathematics Difficulty 39
9.1.3 Relationship between Additional Mathematics Grade and Their Calculus Result 41
9.1.4 Distribution of Calculus Result by Preferred Learning Style 43
9.1.5 Preparation Methods by Gender Distribution 45
9.1.6 Mathematics Difficulty Level and Place of Studying 47
9.1.7 Focus Levels in Mathematics and Their Learning Style 48
9.1.8 Focus Levels in Mathematics and Their Impact on Calculus Results 50

9.2 Numerical Data Analysis 52
9.2.1 Study Hours and Preparation Days vs. Calculus Result 52
9.2.2 The Relationship between Hours of Sleep on Calculus Result 54
9.2.3 The Relationship Between Study Hours and Social Media Usage 55
9.2.4 The Relationship between Preparation Days and Number of Additional Materials 57
9.2.5 The Relationship between Hours of Exercising and Number of Meals 58
9.2.6 The Relationship Between Hours of Sleep, Number of Devices, and Calculus Results 60
9.2.7 Distribution of Number of Housemates by Calculus Result 62
9.2.8 Comparison of Average Study Habits (Pass vs. Fail) 63
9.2.9 Preparation Days and Distance from Home by Calculus Result 65

10.0 RESULT AND DISCUSSION **66**

11.0 CONCLUSION **67**

12.0 RECOMMENDATIONS/LIMITATIONS **67**

REFERENCES **69**

APPENDICES **70**

LIST OF FIGURES

FIGURE **PAGE**

7.1	Data Collection	4
7.2	Data Information	4
7.3	Code Implementation to View the Data Information	5
7.4	Variables of Categorical Data	5
7.5	Code Implementation to View the Categorical Data	5
7.6	Variables of Numerical Data	6
7.7	Code Implementation to View the Numerical Data	6
7.8	Total data for each calculus result	7
7.9	Code Implementation to View the Data for Label	7
7.10	Features column	7
7.11	Code Implementation to View Features Column	8
8.1	Number of Missing Value before Data Cleaning	10
8.2	Code Implementation to Display Missing Values	10
8.3	Number of missing value after data cleaning	11
8.4	Code Implementation to Remove Missing Value	11
8.5	Misclassified Columns	12
8.6	Code Implementation to Transform the Misclassified Columns	12
8.7	Result after Correcting Data Types	12
8.8	All Columns after Completing Data Cleaning	12
8.9	Code Implementation of Data Slicing	14
8.10	Output on Data Slicing	14
8.11	Dataset before Data Transformation	15
8.12	Code Implementation of Data Transformation	15
8.13	Dataset after Data Transformation	16
8.14	Code Implementation of Data Splitting Approach	18
8.15	Implementation of KNN	19
8.16	Code on prediction training and testing sets using KNN	20
8.17	Confusion Matrix and Accuracy Score Visualization for Training Data	21
8.18	Confusion Matrix and Accuracy Score Visualization for Testing Data	21
8.19	Accuracy Scores for Training and Testing Data in default KNN	22
8.20	Implementation of K-Fold Cross-Validation for Data Splitting	23
8.21	Generated Training and Testing Folds of K-Fold Cross-Validation	24
8.22	Implementation of K-Fold Cross-Validation with Hyperparameter Tuning for KNN	26
8.23	Accuracy for all folds and parameter	27
8.24	Code Implementation of average testing accuracy for each parameter combination	28
8.25	Result on average for all parameter combination	28
8.26	Training Phase Using the Best Hyperparameter Combination for KNN	30
8.27	Testing Phase Using the Best Hyperparameter Combination for KNN	31
8.28	Training and Testing Results after using Hyperparameter combination in KNN	32
8.29	Classification Model in Application	33

8.30 Categorical Input Convert to Numerical 34
8.31 Observed Data 35
9.1 Clustered Bar Chart: Internet Interruption vs. Calculus Result for On-Campus and Off-Campus Students 37
9.2 Code Implementation of Internet Interruption vs. Calculus Result for On-Campus and Off-Campus Students 38
9.3 Clustered Bar Chart: Preparation Methods and Perceived Mathematics Difficulty 39
9.4 Code Implementation of Preparation Methods and Perceived Mathematics Difficulty 40
9.5 Heatmap: Relationship between Additional Mathematics Grade and Their Calculus Result 41
9.6 Code Implementation of Relationship between Additional Mathematics Grade and Their Calculus Result 42
9.7 Stacked Bar Chart: Distribution of Calculus Result by Preferred Learning Style 43
9.8 Code Implementation of Distribution of Calculus Result by Preferred Learning Style 44
9.9 Ring Chart: Preparation Methods by Gender Distribution 45
9.10 Code Implementation of Preparation Methods by Gender Distribution 46
9.11 Clustered Bar Chart: Mathematics Difficulty Level and Place of Studying 47
9.12 Code Implementation of Mathematics Difficulty Level and Place of Studying 47
9.13 Clustered Bar Chart: Focus Levels in Mathematics and Their Learning Style 48
9.14 Code Implementation of Focus Levels in Mathematics and Their Learning Style 49
9.15 Stacked Bar Chart: Focus Levels in Mathematics and Their Impact on Calculus Results 50
9.16 Code Implementation of Focus Levels in Mathematics and Their Impact on Calculus Results 51
9.17 Scatter Plot: Study Hours and Preparation Days vs. Calculus Result 53
9.18 Code Implementation of Study Hours and Preparation Days vs. Calculus Result 53
9.19 Boxplot: The Relationship between Hours of Sleep on Calculus Result 54
9.20 Code Implementation of The Relationship between Hours of Sleep on Calculus Result 54
9.21 Scatter Plot with Regression Line: The Relationship Between Study Hours and Social Media Usage 56
9.22 Code Implementation of The Relationship Between Study Hours and Social Media Usage 56
9.23 Scatter Plot with Regression Line: The Relationship between Preparation Days and Number of Additional Materials 57
9.24 Code Implementation of The Relationship between Preparation Days and Number of Additional Materials 58
9.25 2D Density Plot: The Relationship between Hours of Exercising and Number of Meals 59
9.26 Code Implementation of The Relationship between Hours of Exercising and Number of Meals 59
9.27 Line Plot with Confidence Interval: The Relationship Between Hours of Sleep, Number of Devices, and Calculus Results 60

9.28 Code Implementation of The Relationship Between Hours of Sleep, Number of Devices, and Calculus Results 61
9.29 Violin Plot: Distribution of Number of Housemates by Calculus Result 62
9.30 Implementation of Distribution of Number of Housemates by Calculus Result 62
9.31 Radar Chart: Average Study Habits (Pass vs. Fail) 64
9.32 Code Implementation of Average Study Habits (Pass vs. Fail) 64
9.33 Line Graph: Preparation Days and Distance from Home by Calculus Result 65
9.34 Code Implementation of Preparation Days and Distance from Home by Calculus Result 66

LIST OF TABLES

TABLE **PAGE**

8.1	Table Comparison Before and After Data Cleaning	13
8.2	Table Comparison Before and After Data Slicing	14
8.3	Table Comparison Before and After Data Transformation	16
8.4	Training and Testing Datasets for K-Fold Cross-Validation	25
8.5	First Data Input	33
8.6	Second Data Input	34

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
CSC	Computer Science
CSV	Comma-Separated Values
EDM	Educational Data Mining
KF	K-Fold (Cross-validation approach)
KNN	K-Nearest Neighbors
L1	Manhattan Distance (Norm-1 in mathematics)
L2	Euclidean Distance (Norm-2 in mathematics)
ML	Machine Learning
QR	Quick Response (Code)
STEM	Science, Technology, Engineering, and Mathematics
TH	Tidak Hadir (Absent in Malay)

1.0 Introduction

Calculus is an important subject in many university programs particularly in science, technology, engineering, and mathematics (STEM) disciplines. Despite its importance, the subject is considered a challenge to significant population of students who appear to be very difficult in understanding calculus concepts and fail to do well mostly because of that. A survey carried out on 34 university students in UiTM Jasin indicated that 55.9% of such students regard calculus as the inherent most difficult subject to study in their academic career.

Calculus typically brings various dimensions in performance for students, which performance is determined by their prior mathematical knowledge, their study habits, and their overall attitude towards the subject. For instance, a study by Aloyod (2024) found significant correlations between calculus performance and factors such mathematics attitudes, study habits and pre and post-calculus knowledge. In this study, a predictive analysis is performed and will be discussed below.

2.0 Problem Statement

- Many students struggle with calculus. The survey was conducted and among 34 respondents, 55.9% of them choose calculus as the hardest subject in university.
- There is a lack of predictive systems and comprehensive data analysis that identify students' success or failure in calculus.

3.0 Project Objectives

- To design and develop a system using the KNN algorithm to predict students' success or failure in calculus.
- To evaluate the accuracy and effectiveness of the predictive model in identifying at-risk students.
- To perform detailed data analysis and create visualizations that uncover patterns related to the performance of students in calculus.

4.0 Project Scope

This project focuses on developing a predictive system to analyse and predict the university student's success in calculus courses. Over 518 student responses have been collected and will be utilized for prediction purposes. This study examines on various factors that influence calculus performance, including prior academic achievements, study habits, and demographic information. K-Nearest Neighbors (KNN) algorithm is employed to assist in identification of a failing student which could lead to timely intervention and in some other way, boost learning outcomes. The project additionally, involves complete data analysis and visualization to discover patterns and trends associated with success or failure in calculus that can be used as information by educators and institutions.

5.0 Significance of Project

The significance of this project holds the potential for enhancing educational outcomes in calculus courses at Malaysian universities. By analyzing over 518 collected student responses, the system developed can identify students who are at risk of underperforming in calculus. Early identification enables timely intervention such as personalized support and educators can allocate resources on targeted individuals to address the challenges and improve student success rates. Moreover, this data analysis can provide comprehensive insights into factors that influence student performance and also informing educators in their efforts to design effective teaching strategies and curriculum. The implementation of this predictive models has shown to improve academic performance and reduce dropout rates which has been proven by studies in educational data mining (Yağcı, 2022).

6.0 Literature Review

Calculus serves as foundational subject in STEM education in most Malaysian university syllabus yet most student often encounter significant challenges in mastering its concepts. Research indicates that calculus being a difficult subject often linked with inadequate secondary education/high school preparation and insufficiency in understanding prerequisite material which can impede the student's success in university-level calculus courses (Ayebo et al., 2017).

Educational Data Mining (EDM) has emerged and become an essential field dedicated to to analyze educational data to enhance learning outcomes. Data mining techniques employed makes EDM able to uncover patterns and insights which can inform educational practices ultimately combating the failure rates in calculus course (Jawthari & Stoffová, 2021). The importance of predictive analysis lies in using EDM to improve knowledge about predicting - performance and identify - students who are going to fail. For such purposes, machine learning algorithms like k-Nearest Neighbours (KNN) are frequently employed. Machine learning algorithms such as k-Nearest Neighbours (KNN) are commonly used for this purpose. The KNN algorithm is classified as data points based on the majority class among their 'k' nearest neighbours which makes it effective in educational settings for predicting outcomes like student success or failure (Hidayati & Hermawan, 2021).

Additionally, Hidayati & Hermawan (2021) demonstrated the efficacy of KNN algorithm with Euclidean and Manhattan distance to classify students' graduation status and the outcome achieved an outstanding accuracy of 85.28% with k=7. This discovery comes in the light of highlighting the potentiality of the algorithm in prediciting the outcomes from educational systems and suggesting a timely intervention. The expected output combines predictive analysis and machine learning algorithms, like the KNN algorithm, as being a promising opportunity relative to challenges for calculus courses.

7.0 Data Collection

The dataset was collected through a survey conducted using Google Forms, comprising 21 questions and receiving responses from a total of 518 participants. After gathering the data, it was imported by specifying the directory of the comma-separated values (CSV) file. The dataset starts with the first entry at index 2. Figure 7.1 presents the dataset collection. Referring to Figure 7.2, which illustrates the data details, it is observed that the dataset consists of 10 categorical variables and 13 numerical variables. Figure 7.3 shows the code implementation to view the data information in Figure 7.2. The subsequent section will further explore data analysis objectives and data types.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	Timestamp	take_calculus	c_result	c_addmath_gred	c_learning_style	c_how_preparation	c_focus_math	c_difficulty_math	n_hours_studying	n_days_preparation	n_hours_sleep	c_gender	c_on_off_campus	c_place_studying	c_extracurricular	c_internet_interruption	n_add_materials	n_hours_exercising	n_hours_socmed	n_distance	n_housemate	n_meals	n_devices
2	12/5/2024 10:20	c_result	Pass / Lu	A-	Kinesthetic / Kine	Self-study / Belajar	3	3	2	14	6	Male / Lel	On-campus / Luar	At school or univer	Yes / Ya	2	2	4	5	3	8	2	3
3	12/5/2024 10:23	Yes / Ya	Fail / Ga	C+	Visual (e.g., pictu	Group Study / Belaj	2	4	2	7	4	Female / F	On-campus / Dala	At school or univer	Yes / Ya	4	1	1	8	2	8	3	4
4	12/5/2024 10:27	Yes / Ya	Pass / Lu	A+	Kinesthetic / Kine	Group Study / Belaj	4	2	10	7	8	Male / Lel	On-campus / Dala	At home / Di rumа	No / Tidak	2	4	5	3	1	4	2	3
5	12/5/2024 10:27	Yes / Ya	Pass / Lu	A	Kinesthetic / Kine	Group Study / Belaj	4	1	5	30	8	Male / Lel	On-campus / Dala	At school or univer	Yes / Ya	1	4	5	4	1.5	6	3	3
6	12/5/2024 10:28	Yes / Ya	Pass / Lu	B	Visual (e.g., pictu	Self-study / Belajar	3	4	2	5	5	Male / Lel	Off-campus / Luar	At home / Di rumа	No / Tidak	3	3	0	5	10	9	2	4
7	12/5/2024 10:29	Yes / Ya	Pass / Lu	A	Auditory / Pender	Online Courses / Ku	3	2	4	3	7	Female / F	On-campus / Dala	At cafes or other p	Yes / Ya	2	2	3	6	5	6	2	2
8	12/5/2024 10:30	Yes / Ya	Fail / Ga	E	Reading & Writ	Self-study / Belajar	1	5	1	3	3	Male / Lel	Off-campus / Luar	At home / Di rumа	Yes / Ya	3	1	2	10	5	8	2	4
9	12/5/2024 10:30	Yes / Ya	Fail / Ga	D	Reading & Writ	Online Courses / Ku	2	5	3	4	7	Female / F	On-campus / Dala	At a tuition center	Yes / Ya	4	2	2	6	4	6	3	2
10	12/5/2024 10:30	Yes / Ya	Pass / Lu	C	Auditory / Pender	Self-study / Belajar	3	4	5	7	7	Male / Lel	On-campus / Dala	At school or univer	Yes / Ya	2	2	7	6	1	6	2	4
11	12/5/2024 10:31	Yes / Ya	Fail / Ga	E	Visual (e.g., pictu	Self-study / Belajar	1	5	1	1	4	Male / Lel	Off-campus / Luar	At home / Di rumа	Yes / Ya	3	1	2	8	6	6	2	1
12	12/5/2024 10:33	Yes / Ya	Pass / Lu	A-	Kinesthetic / Kine	Group Study / Belaj	2	2	1	3	8	Female / F	Off-campus / Luar	At cafes or other p	Yes / Ya	2	1	2	9	8	4	3	2
13	12/5/2024 10:33	Yes / Ya	Fail / Ga	E	Reading & Writ	Self-study / Belajar	1	5	2	2	5	Male / Lel	On-campus / Dala	At home / Di rumа	Yes / Ya	3	1	6	9	1	6	3	2
14	12/5/2024 10:34	Yes / Ya	Fail / Ga	D	Auditory / Pender	Self-study / Belajar	2	4	2	2	6	Female / F	Off-campus / Luar	At cafes or other p	Yes / Ya	4	1	3	7	6	6	1	2
15	12/5/2024 10:34	Yes / Ya	Pass / Lu	A-	Auditory / Pender	Group Study / Belaj	4	4	8	5	8	Male / Lel	Off-campus / Dala	At school or univer	No / Tidak	2	2	4	5	2	2	3	2
16	12/5/2024 10:35	Yes / Ya	Pass / Lu	B+	Reading & Writ	Group Study / Belaj	4	2	10	10	8	Male / Lel	Off-campus / Luar	At home / Di rumа	No / Tidak	1	5	5	4	10	5	3	3
17	12/5/2024 10:35	Yes / Ya	Fail / Ga	G	Reading & Writ	Tutoring / Tuisyen	1	5	4	2	3	Female / F	On-campus / Dala	At school or univer	Yes / Ya	3	1	0	5	8	5	3	3
18	12/5/2024 10:36	Yes / Ya	Fail / Ga	Not Taken / Tidak	Visual (e.g., pictu	Self-study / Belajar	1	5	2	1	6	Male / Lel	Off-campus / Luar	At home / Di rumа	No / Tidak	4	1	2	12	7	8	2	2
19	12/5/2024 10:36	Yes / Ya	Pass / Lu	A	Reading & Writ	Self-study / Belajar	4	3	5	7	7	Female / F	Off-campus / Luar	At school or univer	Yes / Ya	2	3	3	5	5	4	3	3
20	12/5/2024 10:38	Yes / Ya	Fail / Ga	G	Reading & Writ	Self-study / Belajar	3	5	2	4	5	Male / Lel	On-campus / Dala	At home / Di rumа	Yes / Ya	2	1	2	7	3	5	4	3
21	12/5/2024 10:39	Yes / Ya	Fail / Ga	E	Reading & Writ	Group Study / Belaj	2	4	3	6	7	Female / F	Off-campus / Luar	At a tuition center	Yes / Ya	2	2	2	7	4	3	3	2
22	12/6/2024 21:50	Yes / Ya	Fail / Ga	D	Visual (e.g., pictu	Online Courses / Ku	1	5	1	3	10	Male / Lel	Off-campus / Luar	At school or univer	Yes / Ya	3	2	1	10	4	6	2	3
23	12/6/2024 21:53	Yes / Ya	Fail / Ga	E	Auditory / Pender	Self-study / Belajar	2	5	2	2	5	Female / F	On-campus / Dala	At school or univer	Yes / Ya	4	1	1	8	2	8	3	3
24	12/6/2024 21:53	Yes / Ya	Pass / Lu	A-	Kinesthetic / Kine	Self-study / Belajar	3	2	6	6	7	Male / Lel	On-campus / Dala	At school or univer	Yes / Ya	2	2	5	5	5	4	2	3
25	12/6/2024 21:55	Yes / Ya	Pass / Lu	A	Visual (e.g., pictu	Tutoring / Tuisyen	3	2	6	5	7	Female / F	On-campus / Dala	At a tuition center	No / Tidak	2	3	4	5	1	4	3	3
26	12/6/2024 21:56	Yes / Ya	Pass / Lu	A+	Kinesthetic / Kine	Online Courses / Ku	4	1	7	7	8	Male / Lel	Off-campus / Luar	At cafes or other p	No / Tidak	1	2	6	4	4	5	3	3
27	12/6/2024 21:57	Yes / Ya	Pass / Lu	A+	Kinesthetic / Kine	Group Study / Belaj	4	1	5	30	8	Male / Lel	Off-campus / Dala	At school or univer	Yes / Ya	1	3	3	5	2	2	3	3
28	12/6/2024 21:58	Yes / Ya	Pass / Lu	B+	Reading & Writ	Self-study / Belajar	3	3	4	4	6	Male / Lel	Off-campus / Luar	At cafes or other p	Yes / Ya	3	1	3	6	10	6	3	2
29	12/6/2024 22:00	Yes / Ya	Pass / Lu	B	Reading & Writ	Self-study / Belajar	2	4	2	3	6	Male / Lel	Off-campus / Luar	At cafes or other p	Yes / Ya	2	1	4	6	1	6	2	2
30	12/6/2024 22:02	Yes / Ya	Fail / Ga	D	Auditory / Pender	Self-study / Belajar	1	4	3	3	5	Male / Lel	Off-campus / Luar	At cafes or other p	Yes / Ya	2	2	4	5	6	6	3	1
31	12/6/2024 22:05	Yes / Ya	Fail / Ga	Not Taken / Tidak	Reading & Writ	Self-study / Belajar	2	4	4	3	5	Female / F	On-campus / Dala	At school or univer	Yes / Ya	2	1	3	4	1	6	1	2
32	12/6/2024 22:07	Yes / Ya	Fail / Ga	G	Visual (e.g., pictu	Self-study / Belajar	1	5	2	2	6	Male / Lel	Off-campus / Dala	At home / Di rumа	Yes / Ya	1	1	4	5	5	6	2	2
33	12/6/2024 22:09	Yes / Ya	Fail / Ga	E	Auditory / Pender	Self-study / Belajar	1	4	3	2	5	Female / F	On-campus / Dala	At cafes or other p	Yes / Ya	3	1	2	5	2	6	2	2
34	12/6/2024 22:11	Yes / Ya	Fail / Ga	Not Taken / Tidak	Auditory / Pender	Online Courses / Ku	2	5	3	3	6	Female / F	Off-campus / Luar	At home / Di rumа	Yes / Ya	2	1	3	4	8	6	2	2
35	12/6/2024 22:14	Yes / Ya	Fail / Ga	C	Kinesthetic / Kine	Self-study / Belajar	2	4	2	5	4	Female / F	On-campus / Dala	At school or univer	Yes / Ya	3	2	2	8	2	6	2	4
36	12/6/2024 22:16	Yes / Ya	Pass / Lu	A+	Reading & Writ	Group Study / Belaj	4	1	4	14	8	Male / Lel	On-campus / Dala	At school or univer	Yes / Ya	1	5	3	5	3	2	3	2
37	12/6/2024 22:18	Yes / Ya	Fail / Ga	C+	Visual (e.g., pictu	Self-study / Belajar	2	4	2	3	5	Male / Lel	Off-campus / Luar	At home / Di rumа	Yes / Ya	3	1	1	7	10	10	4	4
38	12/6/2024 22:21	Yes / Ya	Fail / Ga	D	Kinesthetic / Kine	Online Courses / Ku	2	4	2	2	6	Male / Lel	Off-campus / Luar	At home / Di rumа	Yes / Ya	3	2	2	6	8	8	3	3
39	12/6/2024 22:23	Yes / Ya	Fail / Ga	C	Reading & Writ	Self-study / Belajar	2	4	2	1	5	Male / Lel	Off-campus / Luar	At home / Di rumа	Yes / Ya	3	2	2	10	5	4	1	4
40	12/6/2024 22:24	Yes / Ya	Pass / Lu	A	Kinesthetic / Kine	Group Study / Belaj	4	1	5	20	8	Female / F	On-campus / Dala	At school or univer	Yes / Ya	1	3	3	6	2	2	3	2
41	12/6/2024 22:26	Yes / Ya	Fail / Ga	C	Reading & Writ	Self-study / Belajar	1	5	1	3	5	Male / Lel	Off-campus / Luar	At home / Di rumа	Yes / Ya	3	2	2	7	6	6	1	4

Figure 7.1 Data Collection

```
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Timestamp                 518 non-null    object
 1   take_calculus             518 non-null    object
 2   c_result                  500 non-null    object
 3   c_addmath_gred            500 non-null    object
 4   c_learning_style          500 non-null    object
 5   c_how_preparation         500 non-null    object
 6   c_focus_math              500 non-null    float64
 7   c_difficulty_math         500 non-null    float64
 8   n_hours_studying          500 non-null    float64
 9   n_days_preparation        500 non-null    float64
 10  n_hours_sleep             500 non-null    float64
 11  c_gender                  500 non-null    object
 12  c_on_off_campus           500 non-null    object
 13  c_place_studying          500 non-null    object
 14  c_extracurricular         500 non-null    object
 15  c_internet_interruption   500 non-null    float64
 16  n_add_materials           500 non-null    float64
 17  n_hours_exercising        500 non-null    float64
 18  n_hours_socmed            500 non-null    float64
 19  n_distance                500 non-null    float64
 20  n_housemate               500 non-null    float64
 21  n_meals                   500 non-null    float64
 22  n_devices                 500 non-null    float64
dtypes: float64(13), object(10)
```

Figure 7.2 Data Information

```python
import pandas as pd

df = pd.read_csv('C:/Users/aqima/OneDrive/Documents/CSC649 - Special Topics in CS/Project/data.csv')
df.info()
```

Figure 7.3 Code Implementation to View the Data Information

7.1 Data Analytic Purposes

Data analysis explains the main goals of using data such as gaining insights and making better decisions. This section prepares the next parts that focus on methods for analyzing categorical data and numerical data while aligning with the overall goals of data analysis.

7.1.1 Categorical Data

Figure 7.4 shows a list of 10 variables classified as categorical because their data type is 'object.' The 'object' type usually indicates non-numeric information or categorical attributes in the dataset. Figure 7.5 shows the code implementation to view the categorical data.

```
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Timestamp          518 non-null    object
 1   take_calculus      518 non-null    object
 2   c_result           500 non-null    object
 3   c_addmath_gred     500 non-null    object
 4   c_learning_style   500 non-null    object
 5   c_how_preparation  500 non-null    object
 6   c_gender           500 non-null    object
 7   c_on_off_campus    500 non-null    object
 8   c_place_studying   500 non-null    object
 9   c_extracurricular  500 non-null    object
dtypes: object(10)
```

Figure 7.4 Variables of Categorical Data

```python
# 7.1.1 Categorical data
object_columns = df.select_dtypes(include=['object'])
object_columns.info()
```

Figure 7.5 Code Implementation to View the Categorical Data

7.1.2 Numerical Data

Figure 7.6 shows a list of numerical variables identified as 'int64' or 'float64' data types. There are 13 numerical entries in the dataset. These variables represent quantitative data or measurements that consist of integers or floating-point values. Figure 7.7 shows the code implementation to view the numerical data.

```
Data columns (total 13 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   c_focus_math             500 non-null    float64
 1   c_difficulty_math        500 non-null    float64
 2   n_hours_studying         500 non-null    float64
 3   n_days_preparation       500 non-null    float64
 4   n_hours_sleep            500 non-null    float64
 5   c_internet_interruption  500 non-null    float64
 6   n_add_materials          500 non-null    float64
 7   n_hours_exercising       500 non-null    float64
 8   n_hours_socmed           500 non-null    float64
 9   n_distance               500 non-null    float64
 10  n_housemate              500 non-null    float64
 11  n_meals                  500 non-null    float64
 12  n_devices                500 non-null    float64
dtypes: float64(13)
```

Figure 7.6 Variables of Numerical Data

```python
# 7.1.1 Numerical data
numerical_columns = df.select_dtypes(include=['float64'])
numerical_columns.info()
```

Figure 7.7 Code Implementation to View the Numerical Data

7.2 Data Science Purposes

Data science defines specific goals related to its context in the document or framework. This section focuses on two main components: 'Label' and 'Features.' These parts aim to explain the unique roles and importance of these components in data science.

7.2.1 Label

The label represents the target variable or the predicted outcome in a model. It is the key feature in a dataset that is the focus of prediction or analysis. In this dataset the label refers to calculus result (‘c_result’) which includes Pass and Fail. Figure 7.8

presents the total number of data points for results in calculus respectively and Figure 7.9 shows the code implementation to view the data for label.

```
c_result
Pass / Lulus     250
Fail / Gagal     250
```

Figure 7.8 Total data for each calculus result

```
# 7.2.1 Label
print(df['c_result'].value_counts())
```

Figure 7.9 Code Implementation to View the Data for Label

7.2.2 Features

Features are the columns in a dataset that help find patterns or connections to predict the target variable. They are important for training a model because the model learns from them to understand the relationship with the label. This helps make accurate predictions or classifications. Figure 7.10 shows the features used in the classification model and Figure 7.11 shows code implementation to view features column.

```
Data columns (total 20 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   c_addmath_gred           500 non-null    object
 1   c_learning_style         500 non-null    object
 2   c_how_preparation        500 non-null    object
 3   c_focus_math             500 non-null    float64
 4   c_difficulty_math        500 non-null    float64
 5   n_hours_studying         500 non-null    float64
 6   n_days_preparation       500 non-null    float64
 7   n_hours_sleep            500 non-null    float64
 8   c_gender                 500 non-null    object
 9   c_on_off_campus          500 non-null    object
 10  c_place_studying         500 non-null    object
 11  c_extracurricular        500 non-null    object
 12  c_internet_interruption  500 non-null    float64
 13  n_add_materials          500 non-null    float64
 14  n_hours_exercising       500 non-null    float64
 15  n_hours_socmed           500 non-null    float64
 16  n_distance               500 non-null    float64
 17  n_housemate              500 non-null    float64
 18  n_meals                  500 non-null    float64
 19  n_devices                500 non-null    float64
dtypes: float64(13), object(7)
```

Figure 7.10 Features column

```
# 7.2.2 Features

# Drop unused and label columns
df_excluded = df.drop(columns=['c_result', 'Timestamp', 'take_calculus'])

# Display the modified DataFrame
df_excluded.info()
```

Figure 7.11 Code Implementation to View Features Column

Before viewing features column, the unused and label column should be dropped first. It is because, these columns is not represented as a features in a dataset. Figure 7.11 shows that, there are 3 columns should be dropped including 'c_result', 'Timestamp', and 'take_calculus'. By doing that, only features column will be selected and displayed as the output as shown in Figure 7.10.

7.3 Target Respondent

The targeted respondents for this study were students which taking calculus. They may be from high schools or universities where calculus is an important subject. This study collects information about their performance, study habits, past math experience, and other factors that may affect whether they pass or fail. To support the progress of the project a minimum of 500 respondents was needed. Luckily, 518 participants were successfully gathered during the data collection phase. By analyzing this data, the model can find patterns to predict student success. This can help teachers and schools improve learning methods and provide better support for students.

7.3.1 Data Collection Instrument

An online questionnaire was used for data collection because it is easy to create and manage. Google Forms was chosen as the platform because it is accessible and allows team members to work together by editing questions or answer options as needed. The questionnaire QR Code and link is available in the appendices.

7.3.2 Distribution Strategy

The questionnaire was shared through WhatsApp, Instagram, Telegram, and Email to reach as many respondents as possible. Responses were collected from December 5, 2024, until January 5, 2025. The proves will be attached to the appendices.

8.0 Data Science

Data science is an interdisciplinary field that apply scientific methods, algorithms, and systems to extract knowledge and insights from structured and unstructured data. This field integrates domain from statistics, computer science and domain-specific expertise to interpret complex datasets in which can facilitate informed decision-making across various industries. Data science usually involves stages such as data preprocessing, data cleaning, data selection and data transformation. By leveraging all of these techniques, data science enables educators and institutions to uncover hidden patterns and predict future trends

8.1 Data Preprocessing

Data preprocessing encompasses important steps in data set such as data cleaning, data selection, and data transformation. Cleaning entails correcting data quality problems such as duplication and absence of data belonging to relevant features within the data set, changing the data structures or normalizing the features. This subtopic will expound more on data split, selection and transformation.

8.1.1 Data Cleaning

Data Cleaning is the act of amending and removing unwanted parts from the data set to make raw data useful. This process involves the dealing with missing attributes through record deletion or filling them, eliminating redundancies in the data set, and fixing erroneous details for instance spelling mistakes such as misspelled names. There may also exist outliers or noise that have the potential of altering the results, data cleaning also encompasses handling such issues too. By ensuring the data is clean, it will be

suitable to use in any analysis. Figure 8.1 shows the code to display missing values and Figure 8.2 shows the number of missing value for each attributes.

```
Timestamp                   0
take_calculus               0
c_result                   18
c_addmath_gred             18
c_learning_style           18
c_how_preparation          18
c_focus_math               18
c_difficulty_math          18
n_hours_studying           18
n_days_preparation         18
n_hours_sleep              18
c_gender                   18
c_on_off_campus            18
c_place_studying           18
c_extracurricular          18
c_internet_interruption    18
n_add_materials            18
n_hours_exercising         18
n_hours_socmed             18
n_distance                 18
n_housemate                18
n_meals                    18
n_devices                  18
dtype: int64
```

Figure 8.1 Number of Missing Value before Data Cleaning

```
data = "C:/Users/aqima/OneDrive/Documents/CSC649 - Special Topics in CS/Project/data.csv"

# Load the CSV file
df = pd.read_csv(data)

# Before cleaning
print(df.isna().sum())
```

Figure 8.2 Code Implementation to Display Missing Values

There are lots of missing values inside the original dataset. It is because, in our Google Form, there is a question to control the respondents. This question will ensure that only target respondents are allowed to fill the form. Based on Figure 8.1 , there are 18 respondents who were trying to fill out the form and unfortunately, they were not our target respondents. Hence, the missing values need to be removed because it can give a bad impact to our model, specifically the accuracy of the model. Figure 8.3 shows the number of missing value after being removed and Figure 8.4 shows the code implementation to remove the missing value.

```
c_result                    0
c_addmath_gred              0
c_learning_style            0
c_how_preparation           0
c_focus_math                0
c_difficulty_math           0
n_hours_studying            0
n_days_preparation          0
n_hours_sleep               0
c_gender                    0
c_on_off_campus             0
c_place_studying            0
c_extracurricular           0
c_internet_interruption     0
n_add_materials             0
n_hours_exercising          0
n_hours_socmed              0
n_distance                  0
n_housemate                 0
n_meals                     0
n_devices                   0
dtype: int64
```

Figure 8.3 Number of missing value after data cleaning

```
# Remove rows where the specified column contains "No / Tidak"
df = df.drop(df[df['take_calculus'] == 'No / Tidak'].index)

# Drop column timestamp and take_calculus
df = df.drop(columns=['Timestamp', 'take_calculus'])

# After cleaning
print(df.isna().sum())
```

Figure 8.4 Code Implementation to Remove Missing Value

The ‘take_calculus’ column is a control question that have been mentioned just now. If the column has a value ‘No / Tidak’, it means that the respondents did not take calculus on their studies and the row that include the value need to be removed. Other than removing missing values, Figure 8.4 also drop the unwanted column which are ‘Timestamp’ and ‘take_calculus’. These columns are no longer to be used to build our model.

Other than that, there are certain columns which were misclassified in terms of column’s type. Hence, it is important to recorrect the column type, otherwise it can cause errors in certain operations, such as encoding, grouping, or statistical computations. For example, column ‘c_focus_math’, ‘c_difficulty_math’, and ‘c_internet_interruption’ should be a categorical variable like has been shown in Figure 8.5.

```
c_focus_math                float64
c_difficulty_math           float64
c_internet_interruption     float64
dtype: object
```

Figure 8.5 Misclassified Columns

Hence, it is important to transform the misclassified columns into the correct data types. Figure 8.6 shows the code implementation to transform the misclassified columns into the correct one and Figure 8.7 shows the result after correcting their data types. Next, Figure 8.8 shows all columns after completing data cleaning.

```
df['c_focus_math'] = df['c_focus_math'].astype('object')
df['c_difficulty_math'] = df['c_difficulty_math'].astype('object')
df['c_internet_interruption'] = df['c_internet_interruption'].astype('object')
```

Figure 8.6 Code Implementation to Transform the Misclassified Columns

```
c_focus_math                object
c_difficulty_math           object
c_internet_interruption     object
dtype: object
```

Figure 8.7 Result after Correcting Data Types

```
Data columns (total 21 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   c_result                 500 non-null    object
 1   c_addmath_gred           500 non-null    object
 2   c_learning_style         500 non-null    object
 3   c_how_preparation        500 non-null    object
 4   c_focus_math             500 non-null    object
 5   c_difficulty_math        500 non-null    object
 6   n_hours_studying         500 non-null    float64
 7   n_days_preparation       500 non-null    float64
 8   n_hours_sleep            500 non-null    float64
 9   c_gender                 500 non-null    object
 10  c_on_off_campus          500 non-null    object
 11  c_place_studying         500 non-null    object
 12  c_extracurricular        500 non-null    object
 13  c_internet_interruption  500 non-null    object
 14  n_add_materials          500 non-null    float64
 15  n_hours_exercising       500 non-null    float64
 16  n_hours_socmed           500 non-null    float64
 17  n_distance               500 non-null    float64
 18  n_housemate              500 non-null    float64
 19  n_meals                  500 non-null    float64
 20  n_devices                500 non-null    float64
dtypes: float64(10), object(11)
```

Figure 8.8 All Columns after Completing Data Cleaning

Table 8.1 shows the differences before and after data cleaning. Before cleaning, the dataset may have missing values, formatting issues, outliers, and irrelevant data, making it unsuitable for analysis. After cleaning, missing values are handled, inconsistencies are corrected, outliers are addressed, and unnecessary data is removed. This process ensures the dataset is clean, consistent, and ready for analysis or machine learning.

Table 8.1 Table Comparison Before and After Data Cleaning

Aspect	Before Data Cleaning	After Data Cleaning
Missing Data	Contains missing values that can cause errors.	Missing values are filled or removed.
Inconsistent Data	Formatting issues or duplicates.	Data is standardized, and duplicates are removed.
Outliers	Extreme values that may skew results.	Outliers are corrected or removed.
Irrelevant Data	Includes unnecessary or redundant columns.	Irrelevant columns are removed.
Usability	Not ready for analysis or modeling.	Clean and ready for analysis or modeling.

8.1.2 Data Selection

Data selection organizes a dataset for analysis by separating features ('x') and labels ('y'). Labels identify target outcomes, whereas features represent variables. This technique, known as "data slicing," establishes the framework for developing a model. Separating features and labels is crucial for effective model training and analysis during data slicing. Figure 8.9 shows the coding of the data slicing process where 'x' is created as features by taking all columns except 'c_result' through the drop function, and 'y' as a target and takes only the 'c_result' column. Figure 8.10 shows the output after applying data slicing.

```python
# Define features (X) and target (Y)
X = df.drop(columns=['c_result'])  # Drop the target column for features
Y = df['c_result']  # Assign the target column to Y

# Print columns for X and Y
print("Feature columns (X):", X.columns.tolist())  # Print feature column names
print("Target column (Y):", Y.name)  # Print the name of the target column
```

Figure 8.9 Code Implementation of Data Slicing

```
Feature columns (X): ['c_addmath_gred', 'c_learning_style', 'c_how_preparation', 'c_focus_math', 'c_difficulty_math', 'n_hours_study
ing', 'n_days_preparation', 'n_hours_sleep', 'c_gender', 'c_on_off_campus', 'c_place_studying', 'c_extracurricular', 'c_internet_int
erruption', 'n_add_materials', 'n_hours_exercising', 'n_hours_socmed', 'n_distance', 'n_housemate', 'n_meals', 'n_devices']
Target column (Y): c_result
```

Figure 8.10 Output on Data Slicing

Table 8.2 highlights the differences between the dataset structure before and after data slicing, emphasizing the importance of this step in creating a machine-learning-ready dataset.

Table 8.2 Table Comparison Before and After Data Slicing

Aspect	Before Data Slicing	After Data Slicing
Dataset Structure	Contains all columns, including features and the target.	Divided into X (features) and Y (target).
Target Column Included	Target column ('c_result') is part of the dataset.	Target column ('c_result') is stored separately in Y.
Feature Columns Included	Features and target mixed together.	Only feature columns remain in X.
Usability for Modeling	Not ready for machine learning as target is not separated.	Ready for modeling with X as input and Y as target.

The table compares the dataset structure before and after data slicing. Before slicing, the dataset includes all columns, making it unclear which are inputs (features) and which is the output (target). This format is not suitable for machine learning models. After slicing, the dataset is split into two parts: X (features) and Y (target). The features are used as input variables, and the target is used for predictions. This separation prepares the data for machine learning by making it organized and ready for training and analysis.

8.1.3 Data Transformation

Data transformation converts data from one format to another, specifically encoding categorical data into integers. The transformation code using label encoding to represent categorical values as numerical ones. Figure 8.11 shows the dataset before the transformation.

	c_result	c_addmath_gred	c_learning_style	c_how_preparation	c_focus_math	c_difficulty_math	n_hours_studying	n_days_preparation	n_hours_sleep	c_gender	...
0	Pass / Lulus	C	Reading & Writting / Membaca & Menulis	Self-study / Belajar sendiri	3.0	3.0	4.0	7.0	6.0	Female / Perempuan	...
1	Pass / Lulus	A+	Auditory / Pendengaran (e.g., recording)	Group Study / Belajar berkumpulan	4.0	1.0	4.0	21.0	8.0	Female / Perempuan	...
2	Pass / Lulus	B+	Reading & Writting / Membaca & Menulis	Self-study / Belajar sendiri	3.0	5.0	13.0	14.0	5.0	Female / Perempuan	...
3	Pass / Lulus	B+	Reading & Writting / Membaca & Menulis	Self-study / Belajar sendiri	3.0	5.0	12.0	5.0	6.0	Male / Lelaki	...
4	Pass / Lulus	C+	Auditory / Pendengaran (e.g., recording)	Self-study / Belajar sendiri	3.0	5.0	3.0	5.0	7.0	Male / Lelaki	...

5 rows × 21 columns

Figure 8.11 Dataset before Data Transformation

Before transformation, the dataset contains categorical data (text-based labels) that is not directly usable by machine learning models. Hence, the data need to be transformed first before use it. This can be done by using LabelEncoder() the categorical values are replaced with numerical codes. It makes the the data suitable for processing. Figure 8.12 shows the code for applying LabelEncoder() to transform the data and Figure 8.13 shows the result after applying LabelEncoder().

```
# Label encode the categorical variables
var_mod = ['c_result', 'c_addmath_gred', 'c_learning_style', 'c_how_preparation',
           'c_focus_math', 'c_difficulty_math', 'c_gender', 'c_on_off_campus',
           'c_place_studying', 'c_extracurricular', 'c_internet_interruption']

le = LabelEncoder()
for i in var_mod:
    df[i] = le.fit_transform(df[i])

from IPython.display import display

# Display the DataFrame
display(df.head())
```

Figure 8.12 Code Implementation of Data Transformation

	c_result	c_addmath_gred	c_learning_style	c_how_preparation	c_focus_math	c_difficulty_math	n_hours_studying	n_days_preparation	n_hours_s
0	1	6	2	2	2	2	4.0	7.0	
1	1	1	0	0	3	0	4.0	21.0	
2	1	4	2	2	2	4	13.0	14.0	
3	1	4	2	2	2	4	12.0	5.0	
4	1	7	0	2	2	4	3.0	5.0	

5 rows × 21 columns

Figure 8.13 Dataset after Data Transformation

After data transformation is applied during preprocessing, the data is converted into a numerical form that machine learning algorithms can process and understand. This is essential because machines cannot interpret raw textual or categorical data directly. By transforming the data, algorithms are better equipped to analyze patterns, make predictions, and produce accurate results. Table 8.3 shows the key differences between before and after the data transformation.

Table 8.3 Table Comparison Before and After Data Transformation

Aspect	**Before Transformation**	**After Transformation**
Data Type	Text (Categorical)	Numeric
Machine Learning Readiness	Not usable by machine learning models	Fully usable by machine learning models
Consistency	Prone to text-related issues	Consistent numeric format
Processing	Difficult to compute or analyze directly	Easy to analyze and process

The table illustrates the key differences between data before and after transformation. Before transformation, the data typically exists in text or categorical formats, which cannot be directly processed by machine learning algorithms. For instance, columns like ‘c_gender’ or ‘c_result’ may contain values such as "Male / Female" or "Pass / Fail," which need to be converted into numerical representations like 1 and 0. After transformation, the data becomes numeric, consistent, and fully usable by machine learning models. This process ensures compatibility with algorithms, simplifies analysis, and improves overall model performance. By transforming the data, issues such as inconsistencies, skewness, and machine incompatibility are addressed. It enables an efficient and accurate predictive analysis.

8.2 Classification Model

Classification is a method used to predict related information by analyzing the values of a categorical target or class variable (Alnuaimi & Albaldawi, 2024). In machine learning, the classification model is crucial. The process of assigning data points or observations to a predefined class based on their characteristics is called classification.

The classification model section outlines the approach used to develop and evaluate a predictive model for categorical data. It begins with the data split approach, which ensures the dataset is divided appropriately for training and testing purposes. The modeling process involves selecting a suitable machine learning algorithm and analyzing its performance through metrics and testing. To enhance reliability, the cross-validation approach using K-Fold is employed, where multiple experiments are conducted to evaluate the model's consistency and robustness. The best hyperparameter combination from the K-Fold experiments is proposed to optimize the model. Finally, the classification model is remodeled based on the K-Fold results, and its performance is analyzed to ensure it meets the desired accuracy and reliability for the predictive task. This section establishes the foundation for creating an effective and well-validated classification model.

8.2.1 Data Split Approach

The data split approach is a critical step in preparing the dataset for training and testing the classification model. In this process, the dataset is divided into separate subsets to ensure that the model is trained on one portion of the data and tested on another to evaluate its performance on unseen data.

Figure 8.14 illustrates the process of splitting the dataset into training and testing subsets using the train_test_split function. The features and target variables are separated, and the data is divided into training and testing sets in an 80:20 ratio. Additionally, the code combines the split datasets for better inspection and reference. This step is essential to ensure the machine learning model is trained on a diverse subset and evaluated on unseen data for unbiased performance measurement.

```python
data_clean_le = pd.read_csv("C:/Users/aqima/OneDrive/Documents/CSC649 - Special Topics in CS/Project/clean_data_le.csv")

# Data slicing: Features and target
x = data_clean_le.iloc[:, 1:21]  # Features (columns 1 to 20)
y = data_clean_le.iloc[:, 0]  # Target (column 0)

# Split dataset into training and testing sets (e.g., 80-20 split)
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0, shuffle=True)

# Combine features and target for training and testing sets
train = pd.concat([X_train, y_train], axis=1, sort=False)
test = pd.concat([X_test, y_test], axis=1, sort=False)
```

Figure 8.14 Code Implementation of Data Splitting Approach

Based on the Figure 8.14 , the data is split into training and testing sets with a ratio of 80:20. The training set, which comprises 80% of the data, is used to train the machine learning model, while the remaining 20% is reserved for testing the model's performance on unseen data. The features (x) are extracted from columns 1 to 20, representing the independent variables, while the target (y) is taken from column 0, representing the dependent variable. To ensure consistency and reproducibility, a random state of 0 is applied during the split, and shuffling is enabled to randomize the data before splitting. This approach ensures that the training data is diverse and prevents overfitting while enabling an unbiased evaluation of the model's performance on the testing set. For reference, the training and testing datasets are combined back with their respective target variables for easier inspection and debugging.

8.2.2 Modeling Classification Model

In this section, the K-Nearest Neighbors (KNN) algorithm is used to develop the classification model for the dataset. KNN is a supervised machine learning algorithm that classifies data points based on the majority class of their nearest neighbors in the feature space. The model is trained using the training dataset, which comprises 80% of the total data, and evaluated using the testing dataset, which accounts for the remaining 20%. To ensure the model performs effectively, hyperparameters such as the number of neighbors (n_neighbors) are configured, with the default value of 5 being used initially. The performance of the trained model is evaluated using metrics such as accuracy, precision, recall, F1-score, and a confusion matrix. These metrics provide insights into the model's ability to classify data accurately and identify potential areas for improvement. By using KNN, this section demonstrates the construction and evaluation of a classification model that forms the basis for predictive analysis in the project.

8.2.2.1 Machine Learning Algorithm

The machine learning algorithm used in this project for classification is the K-Nearest Neighbors (KNN) algorithm. KNN is a simple, yet effective supervised learning method used for classification tasks. It works by classifying a data point based on the majority class of its nearest neighbors in the feature space. The algorithm calculates the distance such as Euclidean distance between a data point and all other points in the dataset, then selects the k closest points to determine the majority class.

For this project, the KNN algorithm is implemented using the KNeighborsClassifier from the scikit-learn library. The default value of k (number of neighbors) is 5, which means the algorithm considers the 5 nearest neighbors for classification. In this project, k is not explicitly specified, but it defaults to 5, retaining the same behavior. The training process involves fitting the KNN model to the training data, which comprises 80% of the dataset, and then evaluating its performance using the testing set (20%).

By leveraging KNN, the classification model provides a robust and interpretable approach to predicting the target variable based on its similarity to other data points. The simplicity of KNN makes it a strong baseline model for this classification task. Figure 8.15 showcases the code used to implement KNN for this classification task and the name for this project is 'knn'.

```
# Initialize and train the KNN Classifier
knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
```

Figure 8.15 Implementation of KNN

The fit() method is used to train the model on the training dataset (X_train, y_train), allowing it to learn the relationships between the features and the target variable. This step is critical to prepare the model for predicting the target labels of new, unseen data. The trained model will later be evaluated to measure its performance using testing data, as outlined in the subsequent section.

8.2.2.2 Testing and Performance Analysis

After training the K-Nearest Neighbors (KNN) classification model, it is essential to evaluate its performance on the testing dataset to ensure the model's effectiveness and its ability to generalize to unseen data. In this subsection, the testing process involves making predictions on the testing set (X_test) and comparing them to the actual target values (y_test). The performance is assessed using key evaluation metrics, including accuracy, confusion matrix, precision, recall, and F1-score.

Accuracy measures the overall correctness of the model by calculating the percentage of correctly classified instances. The confusion matrix provides a detailed breakdown of true positives, true negatives, false positives, and false negatives, offering insight into the model's performance for each class. The precision and recall metrics evaluate how well the model identifies positive cases, while the F1-score balances precision and recall, providing a single measure of model performance. These metrics are calculated using functions from the sklearn.metrics module. The results of the evaluation provide insights into the model's strengths and limitations, guiding further improvements or optimizations.

After training the K-Nearest Neighbors (KNN) classifier, the model is used to make predictions on both the training and testing datasets. This is done using the predict() function, which applies the trained model to classify data points. The predictions for the training set (y_train_pred) help assess how well the model fits the training data, while the predictions for the testing set (y_test_pred) are used to evaluate the model's performance on unseen data. Figure 8.16 illustrates the code used for making these predictions.

```
# Make predictions for training and testing sets
y_train_pred = knn.predict(X_train)  # Predictions on training set
y_test_pred = knn.predict(X_test)    # Predictions on testing set
```

Figure 8.16 Code on prediction training and testing sets using KNN

As shown in Figure 8.16, the predict() function is applied to the training data (X_train) and testing data (X_test). The resulting predictions (y_train_pred and y_test_pred) are

compared with the actual target values (y_train and y_test) to calculate evaluation metrics such as accuracy, precision, recall, and F1-score.

To evaluate the performance of the KNN classifier on the training dataset, a confusion matrix and accuracy score are calculated. The confusion matrix provides a visual representation of the classifier's predictions compared to the actual labels, highlighting true positives, true negatives, false positives, and false negatives. The accuracy score is calculated to quantify the percentage of correct predictions made by the model. Figure 8.17 and Figure 8.18 shows the code used to compute and visualize these metrics for the training and testing data respectively.

```
# 8.2.2.2 Data train assessment
train_cm = confusion_matrix(y_train, y_train_pred)
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
plt.figure(figsize=(9, 9))
sns.heatmap(train_cm, fmt='d', annot=True, linewidths=.5, square=True, cmap='Blues_r')
plt.ylabel('Actual Label', fontsize=12)
plt.xlabel('Predicted Label', fontsize=12)
plt.title(f"Training: Accuracy Score: {train_accuracy:.2f}%", size=15)
plt.show()
```

Figure 8.17 Confusion Matrix and Accuracy Score Visualization for Training Data

```
train_cm = confusion_matrix(y_train, y_train_pred)
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
plt.figure(figsize=(9, 9))
sns.heatmap(train_cm, fmt='d', annot=True, linewidths=.5, square=True, cmap='Blues_r')
plt.ylabel('Actual Label', fontsize=12)
plt.xlabel('Predicted Label', fontsize=12)
plt.title(f"Training: Accuracy Score: {train_accuracy:.2f}%", size=15)
plt.show()
```

Figure 8.18 Confusion Matrix and Accuracy Score Visualization for Testing Data

Based on the two diagrams above, the output includes the confusion matrix and accuracy scores for both the training and testing datasets. Figure 8.19 shows the output accuracy for both training and testing phase.

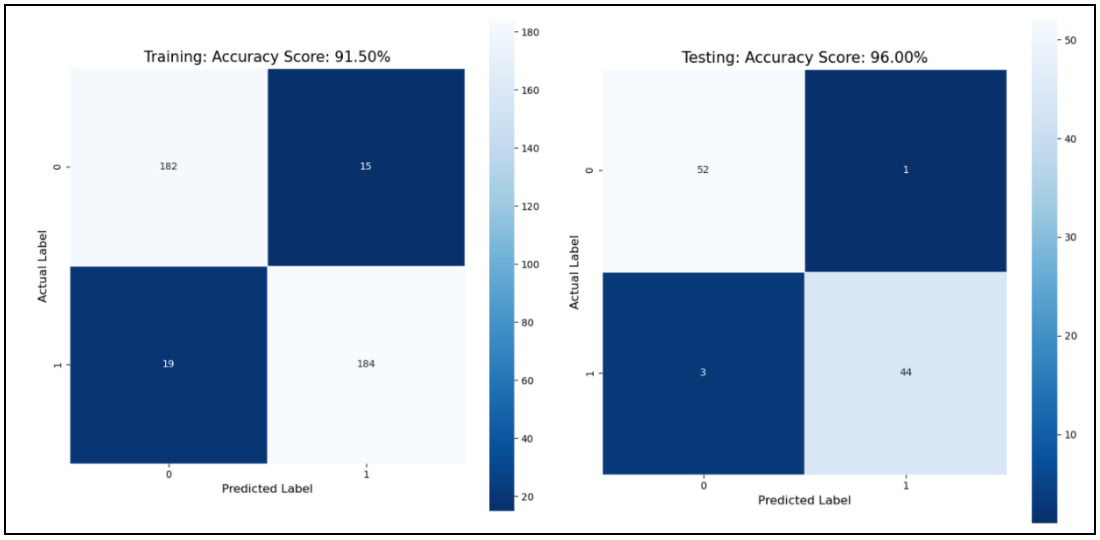


Figure 8.19 Accuracy Scores for Training and Testing Data in default KNN

The confusion matrices and accuracy scores provide an evaluation of the default KNN classifier's performance. The training accuracy of 91.50% reflects that the model has learned well from the training data, while the slightly higher testing accuracy of 96.00% suggests good generalization to unseen data. This difference may indicate that the testing set is slightly easier to classify than the training set, rather than overfitting. However, to further validate and improve the model's performance, this system incorporates the K-Fold Cross-Validation technique. K-Fold ensures the model is evaluated on multiple subsets of the data, reducing the impact of random variations in the training and testing split.

8.2.3 Cross-validation Approach (K-Fold)

Cross-validation is a robust evaluation technique used to assess the generalization ability of a machine learning model by splitting the dataset into multiple subsets, or folds. In this project, the K-Fold Cross-Validation approach is implemented to validate the performance of the KNN classifier. This method divides the dataset into K equal-sized folds, where the model is trained on K-1 folds and tested on the remaining fold. The process is repeated K times, with each fold serving as the testing set once. The final performance metrics are averaged across all folds, ensuring a comprehensive and unbiased evaluation of the model.

By employing K-Fold Cross-Validation, this project aims to address the potential bias introduced by a single train-test split, ensuring the evaluation accounts for the variability in the dataset. Furthermore, the technique is used to identify the optimal hyperparameters, such as the number of neighbors (k) in the KNN model, to achieve the best possible accuracy and generalization.

To implement the K-Fold Cross-Validation technique, the dataset is divided into 4 folds. Each fold is used as a testing set once, while the remaining folds are combined to form the training set. The code in Figure 8.20 Implementation of K-Fold Cross-Validation for Data Splitting demonstrates how the dataset is split and saved into separate files for each fold. This ensures a systematic evaluation of the model's performance across all subsets of the data.

```
df = 'C:/Users/rasdy/Desktop/SEM 5/special topic/clean_data_le.csv'
dataKFold = pd.read_csv(df)
dataKFold.to_csv(clean_data_le, index=False)

# Initialize KFold
kf = KFold(n_splits=4, shuffle=True, random_state=42)  # 4 splits for k-fold

# Create a counter for naming the files
fold_counter = 1

# Loop through the splits
for train_index, test_index in kf.split(dataKFold):
    # Split the data into training and testing sets
    train_data = dataKFold.iloc[train_index]
    test_data = dataKFold.iloc[test_index]

    # Save the training and testing data for this fold to separate files
    train_data.to_csv(f'C:/Users/rasdy/Desktop/SEM 5/special topic/train_fold_{fold_counter}.csv', index=False)
    test_data.to_csv(f'C:/Users/rasdy/Desktop/SEM 5/special topic/test_fold_{fold_counter}.csv', index=False)

    # Increment the fold counter
    fold_counter += 1

print("Operation is done!")
```

```
Operation is done!
```

Figure 8.20 Implementation of K-Fold Cross-Validation for Data Splitting

The code above demonstrates the process of splitting the dataset into 4 folds using the KFold class from the sklearn.model_selection module. The n_splits=4 parameter specifies the number of folds, while shuffle=True ensures that the data is randomized before splitting, and random_state=42 guarantees reproducibility. For each split, the training and testing datasets are extracted and saved as separate files (train_fold_x.csv and test_fold_x.csv).

This systematic approach ensures that all data points are used for both training and testing across different iterations, reducing the bias introduced by a single train-test split. These saved files will later be used for training and evaluating the KNN classifier in each fold, enabling a comprehensive and reliable performance assessment. By leveraging K-Fold Cross-Validation, this project ensures that the model's generalization ability is thoroughly validated.

As part of the K-Fold Cross-Validation process, the dataset is split into 4 folds, with each fold serving as a testing set once while the remaining folds are combined to form the training set. The code used for this operation saves the split datasets as separate files, making it easier to reuse them for training and evaluation. Figure 8.21 shows the structure of the saved files, where each training and testing fold is stored as a distinct CSV file.

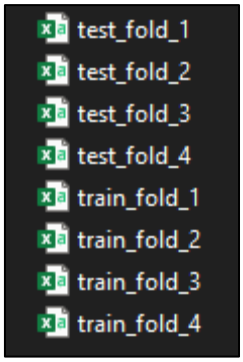


Figure 8.21 Generated Training and Testing Folds of K-Fold Cross-Validation

The files displayed in Figure 8.21 represent the results of the 4-fold cross-validation process. Each pair of training and testing datasets (train_fold_x.csv and test_fold_x.csv) corresponds to one iteration of the cross-validation. This organization ensures that all data points are utilized for both training and testing across the 4 iterations, providing a robust and systematic approach to evaluating the model's performance. These files are subsequently used to train and test the KNN classifier, ensuring consistency and reproducibility in the evaluation process.

8.2.3.1 List of K-Fold Experiments Batch

This subsection provides an overview of the experiment batches created during the K-Fold Cross-Validation process. Each batch corresponds to a single fold, where the dataset is divided into training and testing subsets. This ensures that every data point is used for both training and testing exactly once, offering a comprehensive and unbiased evaluation of the classification model's performance. Table 8.4 lists the datasets generated for each fold during the cross-validation process, showing the separation of training and testing data for all four folds.

Table 8.4 Training and Testing Datasets for K-Fold Cross-Validation

Fold Number	Training Dataset	Testing Dataset
Fold 1	train_fold_1.csv	test_fold_1.csv
Fold 2	train_fold_2.csv	test_fold_2.csv
Fold 3	train_fold_3.csv	test_fold_3.csv
Fold 4	train_fold_4.csv	test_fold_4.csv

Table 8.4 outlines the training and testing datasets created for each fold during the K-Fold Cross-Validation process. Each training dataset (train_fold_x.csv) includes data from 3 folds, while the corresponding testing dataset (test_fold_x.csv) consists of data from the remaining fold. This systematic splitting ensures that all data points are used for both training and testing across the 4 iterations. These files are utilized during the hyperparameter tuning and performance evaluation stages, providing a robust and unbiased assessment of the KNN classifier's accuracy and generalization capabilities.

To enhance the performance of the KNN classifier, K-Fold Cross-Validation was employed alongside hyperparameter tuning. The key parameters tested include the number of neighbors (n_neighbors), weight functions (weights), and algorithm types (algorithm). The experiment evaluated multiple combinations of these parameters across 4 folds to identify the optimal configuration for the model. Figure 8.22 illustrates the code used to iterate through all parameter combinations and calculate the accuracy for both training and testing datasets.

```python
# Define parameter combinations
n_neighbors_values = [3, 5, 7, 9]
weights_values = ['uniform', 'distance']
algorithm_values = ['auto', 'brute', 'ball_tree', 'kd_tree']
parameter_combinations = list(itertools.product(n_neighbors_values, weights_values, algorithm_values))

# Initialize results storage
results = []

# Loop through each fold and test all parameter combinations
for fold in range(1, 5):  # Folds 1 to 4
    # Load train and test datasets for the current fold
    train_data = pd.read_csv(f"C:/Users/aqima/OneDrive/Documents/CSC649 - Special Topics in CS/Project/train_fold_{fold}.csv")
    test_data = pd.read_csv(f"C:/Users/aqima/OneDrive/Documents/CSC649 - Special Topics in CS/Project/test_fold_{fold}.csv")

    # Split features and labels
    X_train, y_train = train_data.iloc[:, 1:], train_data.iloc[:, 0]
    X_test, y_test = test_data.iloc[:, 1:], test_data.iloc[:, 0]

    # Test each parameter combination
    for params in parameter_combinations:
        n_neighbors, weights, algorithm = params

        # Initialize the model
        ModelAKU = KNeighborsClassifier(n_neighbors=n_neighbors, weights=weights, algorithm=algorithm)

        # Train the model
        ModelAKU.fit(X_train, y_train)

        # Predict on the training set
        y_train_pred = ModelAKU.predict(X_train)
        train_accuracy = accuracy_score(y_train, y_train_pred) * 100

        # Predict on the test set
        y_test_pred = ModelAKU.predict(X_test)
        test_accuracy = accuracy_score(y_test, y_test_pred) * 100

        # Format percentages dynamically
        train_accuracy_str = f"{train_accuracy:.2f}%" if train_accuracy % 1 != 0 else f"{int(train_accuracy)}%"
        test_accuracy_str = f"{test_accuracy:.2f}%" if test_accuracy % 1 != 0 else f"{int(test_accuracy)}%"

        # Save the result
        results.append({
            "Fold": fold,
            "n_neighbors": n_neighbors,
            "weights": weights,
            "algorithm": algorithm,
            "Train Accuracy (%)": train_accuracy_str,  # Save as percentage
            "Test Accuracy (%)": test_accuracy_str  # Save as percentage
        })

# Save results to a CSV file in a proper format
results_df = pd.DataFrame(results)
results_df.to_csv("C:/Users/aqima/OneDrive/Documents/CSC649 - Special Topics in CS/Project/accuracy_comparison.csv", index=False)

print("Results saved to 'accuracy_comparison.csv'.")
```

```
Results saved to 'accuracy_comparison.csv'.
```

Figure 8.22 Implementation of K-Fold Cross-Validation with Hyperparameter Tuning for KNN

The accuracy for each combination is computed and formatted dynamically for both the training and testing datasets. The results in Figure 8.23, including parameter values and their respective accuracies, are stored in a list and subsequently converted into a Pandas DataFrame for further analysis. Finally, the results are exported to a CSV file for record-keeping and visualization. This structured approach ensures a thorough evaluation of hyperparameter configurations and facilitates the identification of the optimal parameter combination for the KNN model.

Fold	n_neighbors	weights	algorithm	Train Accuracy (%)	Test Accuracy (%)
1	3	uniform	auto	93.33%	90.40%
1	3	uniform	brute	93.33%	90.40%
1	3	uniform	ball_tree	93.07%	89.60%
1	3	uniform	kd_tree	93.07%	89.60%
1	3	distance	auto	100%	90.40%
1	3	distance	brute	100%	90.40%
1	3	distance	ball_tree	100%	89.60%
1	3	distance	kd_tree	100%	89.60%
1	5	uniform	auto	93.07%	89.60%
1	5	uniform	brute	93.07%	89.60%
1	5	uniform	ball_tree	93.07%	91.20%
1	5	uniform	kd_tree	92.53%	89.60%
1	5	distance	auto	100%	89.60%
1	5	distance	brute	100%	89.60%
1	5	distance	ball_tree	100%	91.20%
1	5	distance	kd_tree	100%	89.60%
1	7	uniform	auto	91.20%	89.60%
1	7	uniform	brute	91.20%	89.60%
1	7	uniform	ball_tree	91.20%	89.60%
1	7	uniform	kd_tree	91.20%	89.60%
1	7	distance	auto	100%	90.40%
1	7	distance	brute	100%	90.40%
1	7	distance	ball_tree	100%	90.40%
1	7	distance	kd_tree	100%	90.40%
1	9	uniform	auto	90.67%	89.60%
1	9	uniform	brute	90.67%	89.60%
1	9	uniform	ball_tree	90.67%	89.60%
1	9	uniform	kd_tree	90.67%	88.80%
1	9	distance	auto	100%	89.60%
1	9	distance	brute	100%	89.60%
1	9	distance	ball_tree	100%	89.60%
1	9	distance	kd_tree	100%	88.80%
2	3	uniform	auto	94.13%	92.80%
2	3	uniform	brute	94.13%	92.80%
2	3	uniform	ball_tree	94.13%	92.80%
2	3	uniform	kd_tree	94.13%	92%
2	3	distance	auto	100%	92.80%
2	3	distance	brute	100%	92.80%
2	3	distance	ball_tree	100%	92.80%
2	3	distance	kd_tree	100%	92%
2	5	uniform	auto	92.53%	92%
2	5	uniform	brute	92.53%	92%
2	5	uniform	ball_tree	92.53%	92%
2	5	uniform	kd_tree	92.53%	92%
2	5	distance	auto	100%	92%
2	5	distance	brute	100%	92%
2	5	distance	ball_tree	100%	92%
2	5	distance	kd_tree	100%	92%
2	7	uniform	auto	92%	92%
2	7	uniform	brute	92%	92%
2	7	uniform	ball_tree	92%	92%
2	7	uniform	kd_tree	92.27%	92%
2	7	distance	auto	100%	92%
2	7	distance	brute	100%	92%
2	7	distance	ball_tree	100%	92%
2	7	distance	kd_tree	100%	92%
2	9	uniform	auto	91.73%	92%
2	9	uniform	brute	91.73%	92%
2	9	uniform	ball_tree	91.73%	92%
2	9	uniform	kd_tree	91.73%	92%
2	9	distance	auto	100%	92%
2	9	distance	brute	100%	92%
2	9	distance	ball_tree	100%	92%
2	9	distance	kd_tree	100%	92%
3	3	uniform	auto	95.20%	91.20%
3	3	uniform	brute	95.20%	91.20%
3	3	uniform	ball_tree	94.67%	91.20%
3	3	uniform	kd_tree	94.67%	90.40%
3	3	distance	auto	100%	91.20%
3	3	distance	brute	100%	91.20%
3	3	distance	ball_tree	100%	91.20%
3	3	distance	kd_tree	100%	90.40%
3	5	uniform	auto	93.33%	91.20%
3	5	uniform	brute	93.33%	91.20%
3	5	uniform	ball_tree	93.07%	91.20%
3	5	uniform	kd_tree	93.07%	91.20%
3	5	distance	auto	100%	91.20%
3	5	distance	brute	100%	91.20%
3	5	distance	ball_tree	100%	91.20%
3	5	distance	kd_tree	100%	91.20%
3	7	uniform	auto	93.07%	90.40%
3	7	uniform	brute	93.07%	90.40%
3	7	uniform	ball_tree	92.80%	90.40%
3	7	uniform	kd_tree	92.80%	90.40%
3	7	distance	auto	100%	90.40%
3	7	distance	brute	100%	90.40%
3	7	distance	ball_tree	100%	90.40%
3	7	distance	kd_tree	100%	90.40%
3	9	uniform	auto	92.27%	90.40%
3	9	uniform	brute	92.27%	90.40%
3	9	uniform	ball_tree	92.27%	90.40%
3	9	uniform	kd_tree	92.27%	90.40%
3	9	distance	auto	100%	90.40%
3	9	distance	brute	100%	90.40%
3	9	distance	ball_tree	100%	90.40%
3	9	distance	kd_tree	100%	90.40%
4	3	uniform	auto	94.40%	88.80%
4	3	uniform	brute	94.40%	88.80%
4	3	uniform	ball_tree	94.13%	89.60%
4	3	uniform	kd_tree	94.13%	89.60%
4	3	distance	auto	100%	88.80%
4	3	distance	brute	100%	88.80%
4	3	distance	ball_tree	100%	89.60%
4	3	distance	kd_tree	100%	89.60%
4	5	uniform	auto	92.53%	88.80%
4	5	uniform	brute	92.53%	88.80%
4	5	uniform	ball_tree	92.53%	88%
4	5	uniform	kd_tree	92.53%	88.80%
4	5	distance	auto	100%	88.80%
4	5	distance	brute	100%	88.80%
4	5	distance	ball_tree	100%	88%
4	5	distance	kd_tree	100%	88.80%
4	7	uniform	auto	92.80%	89.60%
4	7	uniform	brute	92.80%	89.60%
4	7	uniform	ball_tree	92.53%	88.80%
4	7	uniform	kd_tree	92.80%	88.80%
4	7	distance	auto	100%	89.60%
4	7	distance	brute	100%	89.60%
4	7	distance	ball_tree	100%	88.80%
4	7	distance	kd_tree	100%	88.80%
4	9	uniform	auto	92.80%	88.80%
4	9	uniform	brute	92.80%	88.80%
4	9	uniform	ball_tree	92.80%	88.80%
4	9	uniform	kd_tree	92.80%	88.80%
4	9	distance	auto	100%	88.80%
4	9	distance	brute	100%	88.80%
4	9	distance	ball_tree	100%	88.80%
4	9	distance	kd_tree	100%	88.80%

Figure 8.23 Accuracy for all folds and parameter

After the accuracy for all folds and parameter combinations is obtained, the next step involves aggregating the results to identify the optimal hyperparameter combination. This is typically done by calculating the average testing accuracy for each parameter combination across all folds. Averaging ensures that the selected parameters are robust and generalize well across different data splits, rather than being overfit to any particular fold.

Figure 8.24 shows the process of calculating the average testing accuracy for each parameter combination using cross-validation results. The steps include grouping the results by hyperparameter values (n_neighbors, weights, and algorithm) and then computing the average test accuracy for each combination. This ensures the evaluation

of hyperparameters reflects their overall performance across all folds, providing a reliable estimate for selecting the optimal configuration.

```python
# Calculate average testing accuracy for each parameter combination
average_results = (
    results_df.groupby(["n_neighbors", "weights", "algorithm"], as_index=False)
    .agg({"Test Accuracy (%)": lambda x: sum(float(val.strip('%')) for val in x) / len(x)})
)

# Format the average accuracy as a percentage
average_results["Average Test Accuracy (%)"] = average_results["Test Accuracy (%)"].apply(
    lambda x: f"{x:.2f}%" if x % 1 != 0 else f"{int(x)}%"
)

# Drop the intermediate column used for calculation
average_results = average_results.drop(columns=["Test Accuracy (%)"])

# Save average results to a new CSV file
average_results.to_csv(
    "C:/Users/aqima/OneDrive/Documents/CSC649 - Special Topics in CS/Project/average_accuracy_comparison.csv",
    index=False
)

print("Average results saved to 'average_accuracy_comparison.csv'.")
```

Figure 8.24 Code Implementation of average testing accuracy for each parameter combination

Figure 8.25 below shows the result of the average testing accuracy for all parameter combinations derived from the k-fold cross-validation process. The data is grouped by the hyperparameters n_neighbors, weights, and algorithm, and the average testing accuracy is calculated across all folds for each combination. This ensures that the evaluation reflects the overall performance of each parameter combination rather than being biased toward any specific fold.

n_neighbors	weights	algorithm	Average Test Accuracy (%)
3	distance	auto	90.80%
3	distance	ball_tree	90.80%
3	distance	brute	90.80%
3	distance	kd_tree	90.40%
3	uniform	auto	90.80%
3	uniform	ball_tree	90.80%
3	uniform	brute	90.80%
3	uniform	kd_tree	90.40%
5	distance	auto	90.40%
5	distance	ball_tree	90.60%
5	distance	brute	90.40%
5	distance	kd_tree	90.40%
5	uniform	auto	90.40%
5	uniform	ball_tree	90.60%
5	uniform	brute	90.40%
5	uniform	kd_tree	90.40%
7	distance	auto	90.60%
7	distance	ball_tree	90.40%
7	distance	brute	90.60%
7	distance	kd_tree	90.40%
7	uniform	auto	90.40%
7	uniform	ball_tree	90.20%
7	uniform	brute	90.40%
7	uniform	kd_tree	90.20%
9	distance	auto	90.20%
9	distance	ball_tree	90.20%
9	distance	brute	90.20%
9	distance	kd_tree	90%
9	uniform	auto	90.20%
9	uniform	ball_tree	90.20%
9	uniform	brute	90.20%
9	uniform	kd_tree	90%

Figure 8.25 Result on average for all parameter combination

The results indicate that the choice of hyperparameters (n_neighbors, weights, and algorithm) has a marginal impact on the overall accuracy. This suggests that the dataset is well-suited for KNN, and the variations in performance are minor. To further refine the model, additional parameters or evaluation metrics such as precision, recall, or F1-score could be considered depending on the specific requirements of the classification task.

However, to simplify the hyperparameter tuning process and ensure computational efficiency, only three parameters including n_neighbors, weights, and algorithm were chosen for evaluation. Other parameters, such as the distance metric (metric) or the p value for Minkowski distance, were omitted to reduce complexity and avoid overfitting during the tuning process. By focusing on these three core parameters, the tuning process remains computationally feasible while still covering the most impactful factors that influence the model's accuracy.

8.2.3.2 Proposed the Best K-Fold Experiment Batch (The best hyperparameter combination)

The next step involves selecting the parameter combination with the highest accuracy for final model evaluation and deployment. This combination will serve as the optimal configuration for the KNN classifier under the current dataset and evaluation setup.

From the Figure 8.25, it can be observed that the parameter combination of n_neighbors=3, weights='distance', and algorithm='auto' yielded the highest average test accuracy of 90.80%. This indicates that using a smaller number of neighbors (n_neighbors=3) with distance-based weighting allows the KNN model to focus more on closer neighbors, which positively influences its performance. Additionally, the choice of the auto algorithm ensures that the most appropriate search method is selected dynamically based on the dataset's size and structure, contributing to efficient and accurate predictions.

It highlights that other parameter combinations, such as those involving n_neighbors=3 and weights='distance', consistently achieve high accuracy. However, variations in test

accuracy are minimal across different configurations, indicating the robustness of the dataset and the KNN algorithm's adaptability. Based on these results, the selected hyperparameter combination not only maximizes performance but also reflects the best balance between computational efficiency and predictive accuracy.

This optimal combination will be used in the subsequent steps for final model evaluation and deployment to validate its performance on unseen data.

8.2.4 Remodelling Classification Model (Applying K-Fold Result)

Remodelling the classification model involves enhancing the existing model by leveraging lessons from the best-performing K-Fold trial. To increase classification accuracy and performance, this iterative technique attempts to optimize model design and parameters.

Figure 8.26 illustrates the application of the selected hyperparameters derived from the K-Fold cross-validation process to train the final KNN model. The dataset is first loaded and preprocessed, with features (x) and labels (y) extracted and converted into NumPy arrays to ensure compatibility with the KNN classifier. The optimal hyperparameter combination—n_neighbors=3, weights='distance', and algorithm='auto'—is then applied to initialize the KNN model.

```
train_csv = pd.read_csv('C:/Users/rasdy/Desktop/SEM 5/special topic/train.csv')
x = train_csv.iloc[:, 0:20]  # features
y = train_csv.iloc[:, -1]  # label

# Convert dataframe to array
x = x.values
y = y.values
y = y.ravel()

# Initialize and train the model
ModelAKU = KNeighborsClassifier(n_neighbors=3, weights='distance', algorithm='auto')
ModelAKU.fit(x, y)

# Predict on training data
yPred = ModelAKU.predict(x)

# Plot training confusion matrix
plt.figure(figsize=(9, 9))
sns.heatmap(skm.confusion_matrix(y, yPred), fmt= 'd', annot=True, linewidths=.5, square=True, cmap='Blues_r')
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
train_accuracy = skm.accuracy_score(y, yPred) * 100
train_title = f"Training: Accuracy Score: {train_accuracy:.2f}%" if train_accuracy < 100 else "Training: Accuracy Score: 100%"
plt.title(train_title, size=15)
plt.show()
```

Figure 8.26 Training Phase Using the Best Hyperparameter Combination for KNN

```
test_csv = pd.read_csv("C:/Users/rasdy/Desktop/SEM 5/special topic/test.csv")

# Data slicing
x2 = test_csv.iloc[:, 0:20]  # features
y2 = test_csv.iloc[:, -1]  # label

# Convert dataframe to array
x2 = x2.values
y2 = y2.values
y2 = y2.ravel()

# Predict on testing data
yPred2 = ModelAKU.predict(x2)

# Plot testing confusion matrix
plt.figure(figsize=(9, 9))
sns.heatmap(skm.confusion_matrix(y2, yPred2), fmt= 'd', annot=True, linewidths=.5, square=True, cmap='Blues_r')
plt.ylabel('Actual label')
plt.xlabel('Predicted label')
test_accuracy = skm.accuracy_score(y2, yPred2) * 100
test_title = f"Testing: Accuracy Score: {test_accuracy:.2f}%" if test_accuracy < 100 else "Testing: Accuracy Score: 100%"
plt.title(test_title, size=15)
plt.show()
```

Figure 8.27 Testing Phase Using the Best Hyperparameter Combination for KNN

8.2.4.1 Testing and Performance Analysis

Testing and performance analysis is a crucial step in evaluating the effectiveness of a machine learning model. This process involves assessing how well the trained model generalizes to unseen data, which is essential for ensuring its practical utility in real-world scenarios. The primary objective is to measure the model's ability to make accurate predictions on a test dataset that was not used during training.

Figure 8.28 shows the training and testing results for the KNN classifier after applying the best hyperparameter combination identified during the k-fold cross-validation process. The left confusion matrix corresponds to the training phase, where the model achieved an accuracy score of 100%. This indicates that the model perfectly classified all training samples without any misclassifications, suggesting an excellent fit to the training data.

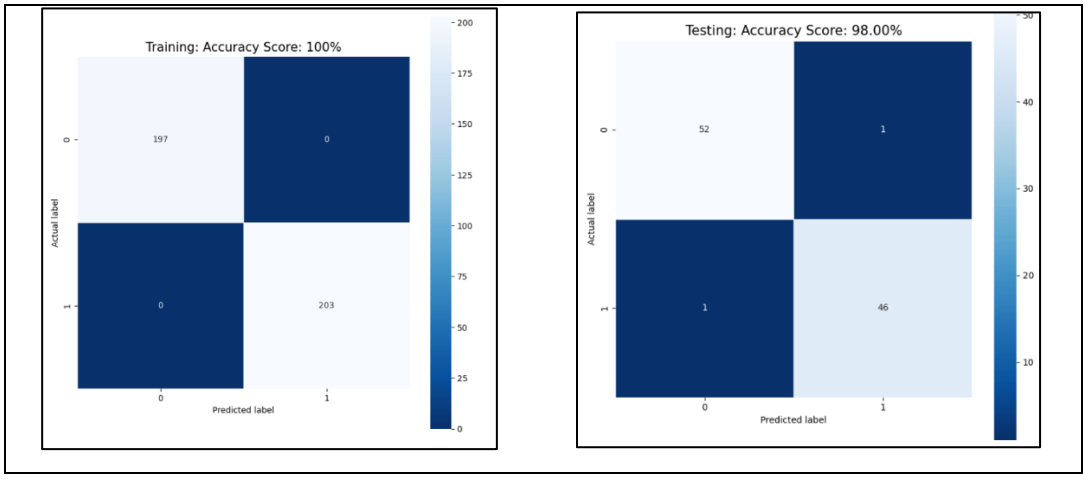


Figure 8.28 Training and Testing Results after using Hyperparameter combination in KNN

It demonstrate the effectiveness of the KNN classifier when configured with the optimal hyperparameters derived from the k-fold cross-validation process. The perfect accuracy achieved during the training phase (100%) reflects the model's ability to learn the patterns in the training data without errors. However, the testing accuracy of 98% ensures that the model generalizes well to unseen data, confirming that it is not overfitting despite the high training accuracy.

8.3 Product (Prototype of application for prediction project)

In addition to the previously outlined objectives, this research paper seeks to explore student performance in Calculus, focusing on its impact on pass or fail rates. The study aims to understand how various factors, including teaching methods, study behaviours, and evaluation techniques, influence students' success and their overall experiences in learning Calculus.

8.3.1 Applying the Classification Model in Application

Integrating the classification model into an application requires the implementation of code, as illustrated in Figure 8.29. The array used in the code serves as the data input for the application.

```
newData[0] = [[10,2,2,0,4,2,2,6,0,0,1,0,0,1,2,9,6,2,3,4]]
newData[1] = [[0,3,3,2,1,5,6,6,1,1,3,0,0,1,2,3,1,3,4,2]]

for i in range(2):
    predictionResult[i] = ModelAKU.predict(newData[i])

    if(predictionResult[i] == 1):
        print(f"Prediction {i+1} result = Pass")
    else:
        print(f"Prediction {i+1} result = Fail")
```

Figure 8.29 Classification Model in Application

8.3.2 Run Application Testing

This section evaluates the application prototype by detailing the data inputs, documenting observations during execution, and analyzing the resulting outcomes.

8.3.2.1 List of Data Input

Table 8.5 and Table 8.6 below show a list of data input to run the application.

Table 8.5 First Data Input

Column	Value
c_addmath_gred	G
c_learning_style	Reading & Writting / Membaca & Menulis
c_how_preparation	Self-study / Belajar sendiri
c_focus_math	1
c_difficulty_math	5
n_hours_studying	2
n_days_preparation	2
n_hours_sleep	6
c_gender	Female / Perempuan
c_on_off_campus	Off-campus / Luar kampus
c_place_studying	At cafes or other public spaces / Kafe atau tempat umum lain
c_extracurricular	No / Tidak
c_internet_interruption	1
n_add_materials	1
n_hours_exercising	2

n_hours_socmed	9
n_distance	6
n_housemate	2
n_meals	3
n_devices	4

Table 8.6 Second Data Input

Column	Value
c_addmath_gred	A
c_learning_style	Visual (e.g., picture)
c_how_preparation	Tutoring / Tuisyen
c_focus_math	3
c_difficulty_math	2
n_hours_studying	5
n_days_preparation	6
n_hours_sleep	6
c_gender	Male / Lelaki
c_on_off_campus	On-campus / Dalam kampus
c_place_studying	At school or university / Di sekolah atau universiti (e.g., classroom, library)
c_extracurricular	No / Tidak
c_internet_interruption	1
n_add_materials	1
n_hours_exercising	2
n_hours_socmed	3
n_distance	1
n_housemate	3
n_meals	4
n_devices	2

All categorical input will be converted into numerical, as shown in Figure 8.30.

```
newData[0] = [[10,2,2,0,4,2,2,6,0,0,1,0,0,1,2,9,6,2,3,4]]
newData[1] = [[0,3,3,2,1,5,6,6,1,1,3,0,0,1,2,3,1,3,4,2]]
```

Figure 8.30 Categorical Input Convert to Numerical

8.3.2.2 List of Observation Results

The observed data will be presented in Figure 8.31.

```
Prediction 1 result = Fail
Prediction 2 result = Pass
```

Figure 8.31 Observed Data

As displayed in Figure 8.31, the application evaluated two sets of inputs and categorized the results accordingly. For Prediction 1, the result is "Fail," indicating that the input parameters suggest the student may not meet the criteria for passing. In contrast, Prediction 2 is classified as "Pass," reflecting that the input data meets the conditions associated with successful performance. These results demonstrate the application's ability to analyze and classify outcomes based on the given data, providing a clear distinction between passing and failing scenarios. The observed results will be critical for further evaluation of the model's effectiveness in accurately predicting academic outcomes in the Calculus subject.

8.3.2.3 Analysis and Discussion

The analysis and discussion phase evaluates the outcomes of the application's predictions for student performance in the Calculus subject, focusing on both students predicted to pass and those predicted to fail. Based on the input data, which may include factors such as study habits, preparation hours, focus on mathematical concepts, and previous calculus performance, the application identified distinct outcomes for two students. One student was predicted to fail, indicating that their input data likely aligned with patterns associated with students who previously struggled in Calculus. This suggests that the student's preparation hours, difficulty in understanding mathematical concepts, or other factors may not meet the necessary thresholds for success. On the other hand, the other student was predicted to pass, reflecting that their input data aligned with characteristics typically observed in successful students. Factors such as adequate study hours, effective preparation strategies, and better grasp of mathematical concepts likely contributed to this positive prediction.

The distinction between pass and fail predictions highlights the application's ability to classify students effectively, providing valuable insights into the conditions that influence success in Calculus. However, it is essential to consider whether all relevant factors have been included in the model. While academic indicators like preparation hours and past grades play a significant role, external influences such as access to resources, motivation levels, and attendance in supplementary classes may also impact performance. To enhance the reliability of predictions, further validation against real-world student outcomes is necessary. Moreover, incorporating additional features and refining the model could improve its accuracy and generalizability. By addressing both pass and fail outcomes, this analysis emphasizes the importance of using predictive tools to identify at-risk students while also recognizing the factors that contribute to success, enabling educators to provide targeted support and encouragement to all students.

9.0 Data Analysis and Visualization

The Data Analysis and Visualization section presents insights from categorical and numerical data using charts such as bar graphs, pie charts, scatter plots, and heatmaps. These charts reveal patterns in study habits, learning styles, preparation time, and external factors like internet interruptions and extracurricular activities, which collectively highlight key contributors to student success or failure in calculus.

9.1 Categorical Data Analysis

Categorical data consists of variables that represent distinct groups or categories. These variables are qualitative rather than quantitative and are typically used to describe characteristics or attributes of a dataset. Categorical data does not have a natural order or numerical meaning, which makes it distinct from numerical data. Examples include gender, preparation methods, and scale of internet interruption.

9.1.1 Internet Interruption vs. Calculus Result for On-Campus and Off-Campus Students

Figure 9.1 compares the academic outcomes (Pass/Fail) of on-campus and off-campus students based on their internet interruption frequency (Rare, Occasional, Frequent, Very Frequent). The data highlights how internet stability impacts academic performance, especially in environments where consistent online access is necessary for learning. The purpose of this analysis is to uncover patterns that can guide interventions for students facing connectivity challenges. Figure 9.2 shows the code on how to implement this visualization.

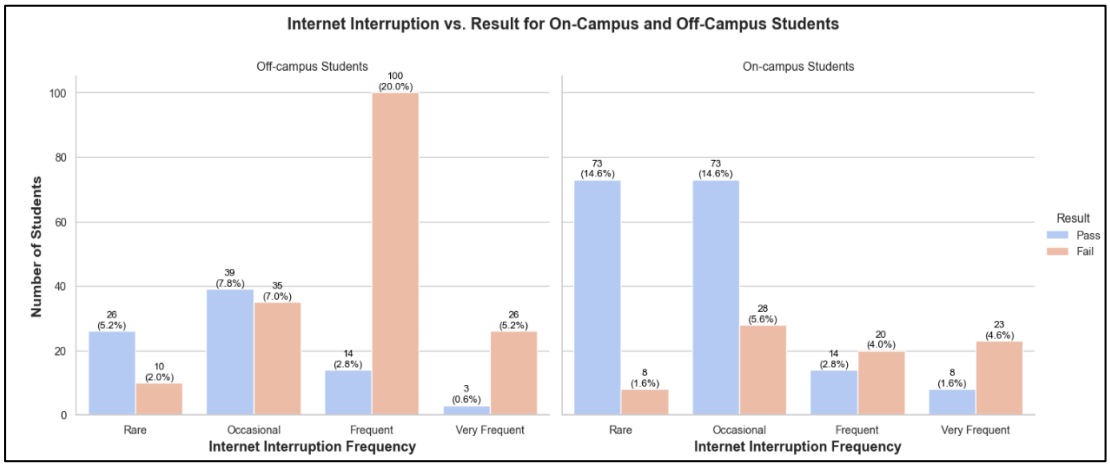


Figure 9.1 Clustered Bar Chart: Internet Interruption vs. Calculus Result for On-Campus and Off-Campus Students

```python
# Load the dataset from the specified path
file_path = "C:/Users/aqima/OneDrive/Documents/CSC649 - Special Topics in CS/Project/clean_data.csv"

# Load the data and replace scales with suitable words
try:
    df_real = pd.read_csv(file_path)

    # Replace scale values with descriptive words (for 1-4)
    scale_mapping = {
        1: "Rare",
        2: "Occasional",
        3: "Frequent",
        4: "Very Frequent"
    }
    df_real["c_internet_interruption"] = df_real["c_internet_interruption"].replace(scale_mapping)

    # Replace "On-campus / Dalam kampus" with "On-campus" and "Off-campus / Luar kampus" with "Off-campus"
    df_real["c_on_off_campus"] = df_real["c_on_off_campus"].replace({
        "On-campus / Dalam kampus": "On-campus",
        "Off-campus / Luar kampus": "Off-campus"
    })

    # Replace "Pass / Lulus" with "Pass" and "Fail / Gagal" with "Fail"
    df_real["c_result"] = df_real["c_result"].replace({
        "Pass / Lulus": "Pass",
        "Fail / Gagal": "Fail"
    })

    # Sort the c_internet_interruption column for consistent ordering
    df_real["c_internet_interruption"] = pd.Categorical(
        df_real["c_internet_interruption"],
        categories=["Rare", "Occasional", "Frequent", "Very Frequent"],
        ordered=True
    )

    # FacetGrid for a combined view of On-Campus and Off-Campus students
    g = sns.catplot(
        data=df_real,
        x="c_internet_interruption",
        hue="c_result",
        col="c_on_off_campus",
        kind="count",
        palette="coolwarm",
        height=6,
        aspect=1.2
    )

    # Add percentages and counts to the bars
    def add_percentages_and_counts(ax, total):
        """Adds percentage labels and counts to the bars in a countplot."""
        for p in ax.patches:
            if p.get_height() > 0:  # Only add labels if the bar height is greater than 0
                percentage = f'{100 * p.get_height() / total:.1f}%'
                count = int(p.get_height())
                ax.annotate(
                    f'{count}\n({percentage})',
                    (p.get_x() + p.get_width() / 2., p.get_height()),
                    ha='center', va='center',
                    fontsize=10, color='black',
                    xytext=(0, 10), textcoords='offset points'
                )

    # Apply annotations to each subplot
    total_students = len(df_real)
    for ax in g.axes.flat:
        add_percentages_and_counts(ax, total_students)

    # Titles and labels
    g.set_titles("{col_name} Students")
    g.set_axis_labels("Internet Interruption Frequency", "Number of Students", fontsize=14, fontweight='bold')
    g.set_xticklabels(rotation=0)
    g._legend.set_title("Result")
    plt.subplots_adjust(top=0.85)
    plt.suptitle("Internet Interruption vs. Result for On-Campus and Off-Campus Students", fontsize=16, fontweight='bold')

    plt.show()

except FileNotFoundError:
    print("File not found. Please check the file path and ensure the file exists.")
except Exception as e:
    print(f"An error occurred: {e}")
```

Figure 9.2 Code Implementation of Internet Interruption vs. Calculus Result for On-Campus and Off-Campus Students

The analysis reveals that frequent and very frequent internet interruptions are strongly associated with higher failure rates, particularly for off-campus students, where failure rates peak at 20%. On-campus students perform better across all categories, likely due to more reliable internet and support. These findings suggest that stable internet access is crucial for academic success.

To address these challenges, educators can focus on providing resources like offline learning materials, access to campus facilities, and additional support for students with connectivity issues. For students, utilizing stable on-campus internet and developing

contingency plans, such as preloading materials, can help mitigate the impact of interruptions on their studies.

9.1.2 Preparation Methods and Perceived Mathematics Difficulty

The preparation methods students use to study for calculus are critical in shaping their perception of the subject's difficulty. Understanding the correlation between various preparation strategies and the perceived difficulty levels can help identify effective approaches that ease the learning process. Figure 9.3 presents a comparative analysis of students' preferred preparation methods, categorized as Self-study, Group Study, Tutoring, and Online Courses, against their perception of mathematics difficulty, ranging from Very Easy to Very Difficult. The inclusion of student counts above each bar highlights the popularity and effectiveness of each method, providing a comprehensive understanding of student preferences. Figure 9.4 shows the code on how to implement this visualization.

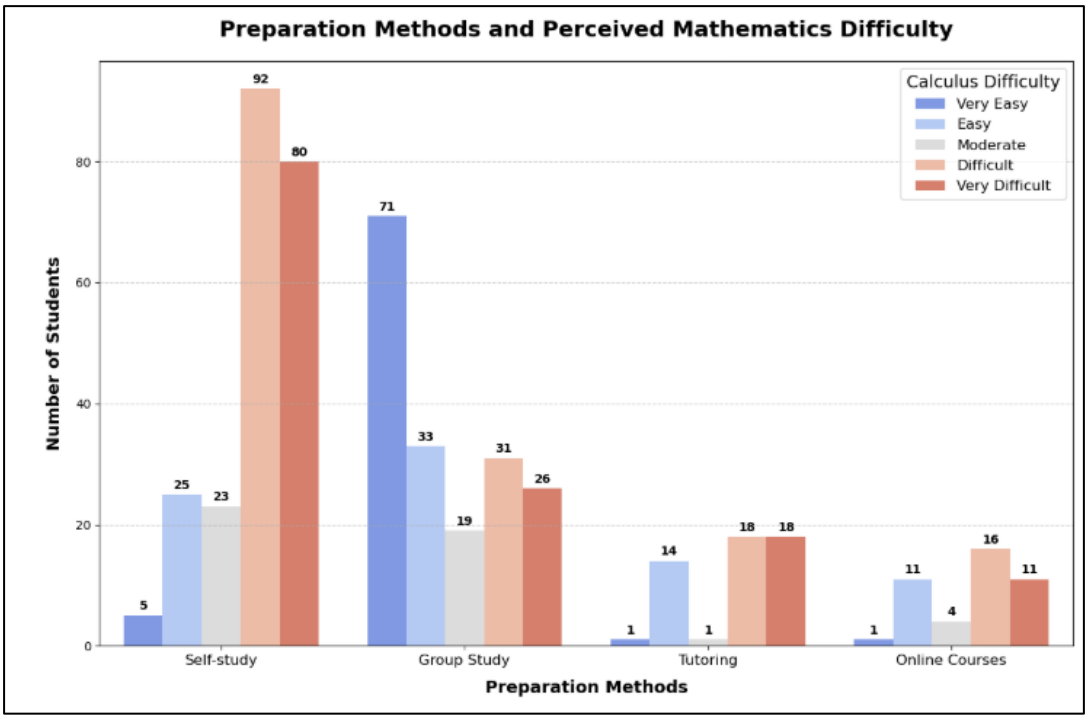


Figure 9.3 Clustered Bar Chart: Preparation Methods and Perceived Mathematics Difficulty

```python
# Map preparation methods to English only
preparation_mapping = {
    'Self-study / Belajar sendiri': 'Self-study',
    'Group Study / Belajar berkumpulan': 'Group Study',
    'Tutoring / Tuisyen': 'Tutoring',
    'Online Courses / Kursus dalam talian': 'Online Courses'
}
data['c_how_preparation'] = data['c_how_preparation'].replace(preparation_mapping)

# Map difficulty levels to descriptive words
difficulty_mapping = {
    1.0: "Very Easy",
    2.0: "Easy",
    3.0: "Moderate",
    4.0: "Difficult",
    5.0: "Very Difficult"
}
data['c_difficulty_math'] = data['c_difficulty_math'].replace(difficulty_mapping)

# Set the order for difficulty levels
difficulty_order = ["Very Easy", "Easy", "Moderate", "Difficult", "Very Difficult"]

# Create the bar plot with enhanced readability
plt.figure(figsize=(12, 8))  # Increased figure size for clarity
ax = sns.countplot(
    data=data,
    x='c_how_preparation',
    hue='c_difficulty_math',
    palette='coolwarm',
    hue_order=difficulty_order  # Ensure proper order of difficulty levels
)

# Title and labels
plt.title('Preparation Methods and Perceived Mathematics Difficulty', fontsize=18, fontweight='bold', pad=20)
plt.xlabel('Preparation Methods', fontsize=14, fontweight='bold', labelpad=10)
plt.ylabel('Number of Students', fontsize=14, fontweight='bold', labelpad=10)

# Add gridlines for better readability
plt.grid(axis='y', linestyle='--', alpha=0.7)  # Grid on Y-axis with dashed lines

# Add number of students above each bar
for bar in ax.patches:
    count = int(bar.get_height())
    if count > 0:  # Only add labels for non-zero bars
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.5,  # Slightly above the bar
            f'{count}',
            ha='center', va='bottom', fontsize=10, color='black', weight='bold'
        )

# Adjust X-axis ticks for readability
plt.xticks(rotation=0, ha='center', fontsize=12)

# Adjust legend for better positioning and readability
plt.legend(
    title='Calculus Difficulty',
    fontsize=12,
    title_fontsize=14,
    loc='upper right'
)

# Improve layout for better spacing
plt.tight_layout()

# Show the plot
# plt.show()
```

Figure 9.4 Code Implementation of Preparation Methods and Perceived Mathematics Difficulty

The diagram illustrates that Self-study and Group Study are the most commonly adopted preparation methods, with Self-study showing a higher tendency towards students perceiving calculus as "Difficult" or "Very Difficult." Conversely, students engaging in Group Study reported a broader spectrum of difficulty levels, indicating that collaborative learning might help alleviate the perceived difficulty of mathematics. Tutoring and Online Courses are less popular preparation methods but are evenly distributed across various difficulty levels, suggesting these methods might cater to specific student needs.

This analysis aids educators in understanding which preparation methods are more effective in reducing students' perceived difficulty in mathematics. For instance, promoting group study sessions could potentially create a supportive environment where students can exchange knowledge and enhance their understanding. Additionally, institutions might consider integrating more structured online resources or tutoring sessions to assist students who struggle with self-directed learning. This insight enables tailored interventions to improve learning outcomes for calculus students.

9.1.3 Relationship between Additional Mathematics Grade and Their Calculus Result

Figure 9.5 illustrates the relationship between students' grades in Additional Mathematics and their results in Calculus (Pass/Fail). By comparing performance percentages, the visualization reveals how prior knowledge in Additional Mathematics influences success in Calculus. Grades range from A+ to G, including cases where students did not take or were absent from Additional Mathematics. The purpose of this analysis is to highlight the predictive value of Additional Mathematics performance in determining outcomes in Calculus. Figure 9.6 shows the code on how to implement this visualization.

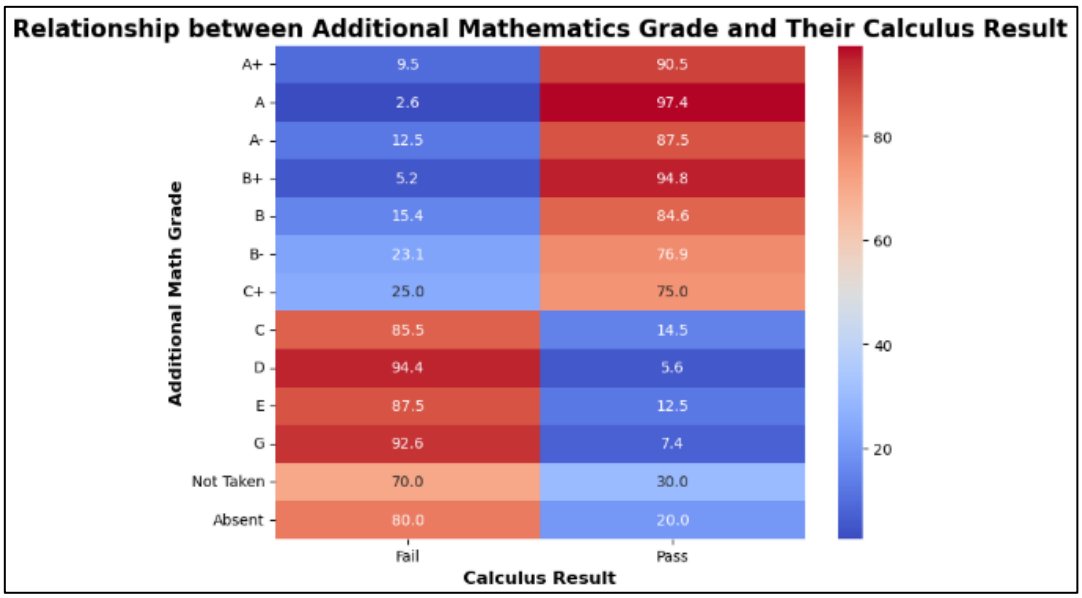


Figure 9.5 Heatmap: Relationship between Additional Mathematics Grade and Their Calculus Result

```python
# Define the desired order for the grades in English
grade_order = [
    "A+", "A", "A-", "B+", "B", "B-", "C+", "C", "D", "E", "G",
    "Not Taken", "Absent"
]

# Define the desired mapping for results
result_mapping = {
    "Pass / Lulus": "Pass",
    "Fail / Gagal": "Fail"
}

# Create a mapping dictionary to convert grades to English
grade_mapping = {
    "A+": "A+",
    "A": "A",
    "A-": "A-",
    "B+": "B+",
    "B": "B",
    "B-": "B-",
    "C+": "C+",
    "C": "C",
    "D": "D",
    "E": "E",
    "G": "G",
    "Not Taken / Tidak Ambil": "Not Taken",
    "TH / Tidak Hadir": "Absent"
}

clean_data = 'C:/Users/aqima/OneDrive/Documents/CSC649 - Special Topics in CS/Project/clean_data.csv'
data_final = pd.read_csv(clean_data)

# Map the 'c_addmath_gred' column to English labels
data_final['c_addmath_gred'] = data_final['c_addmath_gred'].map(grade_mapping)

# Map the 'c_result' column to English labels
data_final['c_result'] = data_final['c_result'].map(result_mapping)

# Convert 'c_addmath_gred' to a categorical variable with the specified order
data_final['c_addmath_gred'] = pd.Categorical(
    data_final['c_addmath_gred'], categories=grade_order, ordered=True
)

# Create the heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(
    pd.crosstab(data_final['c_addmath_gred'], data_final['c_result'], normalize='index') * 100,
    annot=True, cmap='coolwarm', fmt=".1f"
)
plt.title("Relationship between Additional Mathematics Grade and Their Calculus Result", fontsize=16, fontweight='bold')
plt.xlabel("Calculus Result", fontsize=12, fontweight='bold')
plt.ylabel("Additional Math Grade", fontsize=12, fontweight='bold')
plt.show()
```

Figure 9.6 Code Implementation of Relationship between Additional Mathematics Grade and Their Calculus Result

The heatmap reveals a strong correlation between high school Additional Mathematics grades and university Calculus results. Students with higher grades in Additional Mathematics, such as A+ and A, have a high likelihood of passing Calculus (90.5% and 97.4%, respectively). Conversely, those with lower grades like C, D, or G exhibit significantly higher failure rates, exceeding 85%. Notably, students who did not take or were absent from Additional Mathematics show poor performance, with pass rates of only 30% and 20%, respectively. This highlights the critical role of foundational knowledge in determining success in advanced university courses like Calculus.

This analysis helps educators identify students at risk of failing Calculus based on their Additional Mathematics grades. Universities can use this insight to implement targeted interventions, such as remedial math courses, preparatory workshops, or personalized support for students with weaker backgrounds in mathematics. Additionally, students can be encouraged to strengthen their math fundamentals before beginning university, ensuring they are better equipped for the challenges of Calculus.

9.1.4 Distribution of Calculus Result by Preferred Learning Style

Figure 9.7 illustrates the relationship between students' preferred learning styles (Auditory, Kinesthetic, Reading & Writing, and Visual) and their Calculus results (Pass/Fail). By analyzing the number of respondents for each learning style and their outcomes, this visualization aims to uncover trends in how learning preferences may influence academic performance in Calculus. The objective is to provide insights into the effectiveness of different learning approaches and identify areas for improvement. Figure 9.8 shows the code on how to implement this visualization.

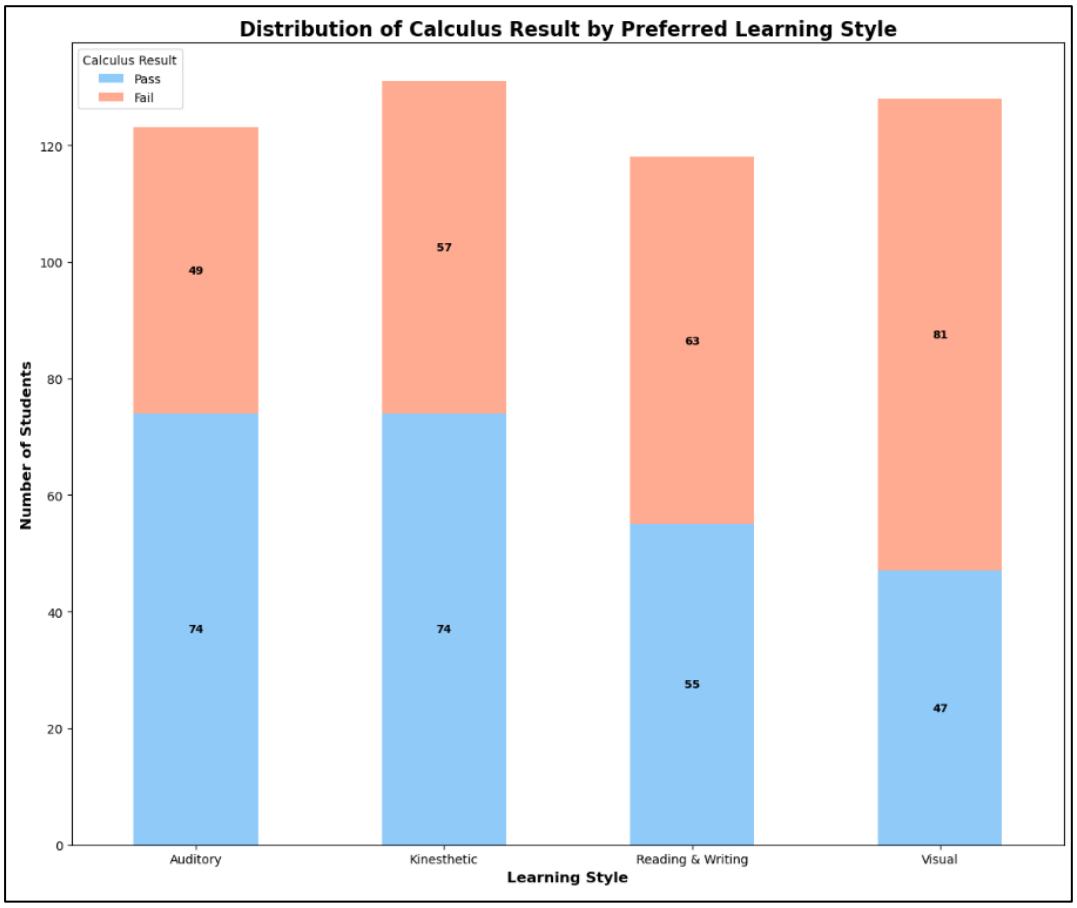


Figure 9.7 Stacked Bar Chart: Distribution of Calculus Result by Preferred Learning Style

```python
learning_style_mapping = {
    "Visual (e.g., picture)": "Visual",
    "Auditory / Pendengaran (e.g., recording)": "Auditory",
    "Kinesthetic / Kinestetik (e.g., apply hands-on)": "Kinesthetic",
    "Reading & Writting / Membaca & Menulis": "Reading & Writing"
}
data["c_learning_style"] = data["c_learning_style"].replace(learning_style_mapping)

# Grouping data by learning style and result
learning_style_result = data.groupby(['c_learning_style', 'c_result']).size().unstack(fill_value=0)

# Creating a stacked bar chart
fig, ax = plt.subplots(figsize=(12, 10))
bars = learning_style_result.plot(kind='bar', stacked=True, ax=ax, color=['#90CAF9', '#FFAB91'])

# Adding data labels
for bar in bars.patches:
    if bar.get_height() > 0:  # Add labels only for non-zero bars
        ax.text(
            bar.get_x() + bar.get_width() / 2,  # Center of the bar
            bar.get_y() + bar.get_height() / 2,  # Center of the bar segment
            f"{int(bar.get_height())}",  # Data value
            ha='center', va='center', fontsize=9, color='black', weight='bold'
        )

# Adding labels, title, and legend
ax.set_title("Distribution of Calculus Result by Preferred Learning Style", fontsize=16, fontweight='bold')
ax.set_xlabel("Learning Style", fontsize=12, fontweight='bold')
ax.set_ylabel("Number of Students", fontsize=12, fontweight='bold')
ax.legend(title="Calculus Result", labels=["Pass", "Fail"], fontsize=10)
plt.xticks(rotation=0, ha='center')

# Enhancing layout
plt.tight_layout()
plt.show()
```

Figure 9.8 Code Implementation of Distribution of Calculus Result by Preferred Learning Style

The analysis reveals significant variations in Calculus results across different learning styles. Students with a Kinesthetic preference show balanced performance, with 74 passing and 57 failing. However, students with a Visual learning preference exhibit a higher failure rate, with 81 failing compared to only 47 passing. Conversely, Auditory learners show a better balance, with an equal number of passes (74) and relatively fewer failures (49). Reading & Writing learners demonstrate the highest failure rate, with 63 failing and only 55 passing. This trend highlights how some learning styles may align better with traditional Calculus teaching methods.

These findings can help educators tailor their teaching strategies to accommodate diverse learning styles. For instance, incorporating more hands-on activities and visual aids could benefit Kinesthetic and Visual learners, while Reading & Writing learners may need additional support through structured notes and text-based resources. For students, this analysis emphasizes the importance of adapting their study strategies to complement their natural preferences, ensuring a more effective learning experience in Calculus.

9.1.5 Preparation Methods by Gender Distribution

To understand the gender distribution in the adoption of different preparation methods for exams, this visualization categorizes data by preparation methods and gender. It highlights the proportion of students using specific methods like self-study, group study, online courses, and tutoring. Figure 9.9 aims to reveal preferences across genders and their potential implications for targeted academic support. shows the code on how to implement this visualization. Figure 9.10 shows the code on how to implement this visualization.

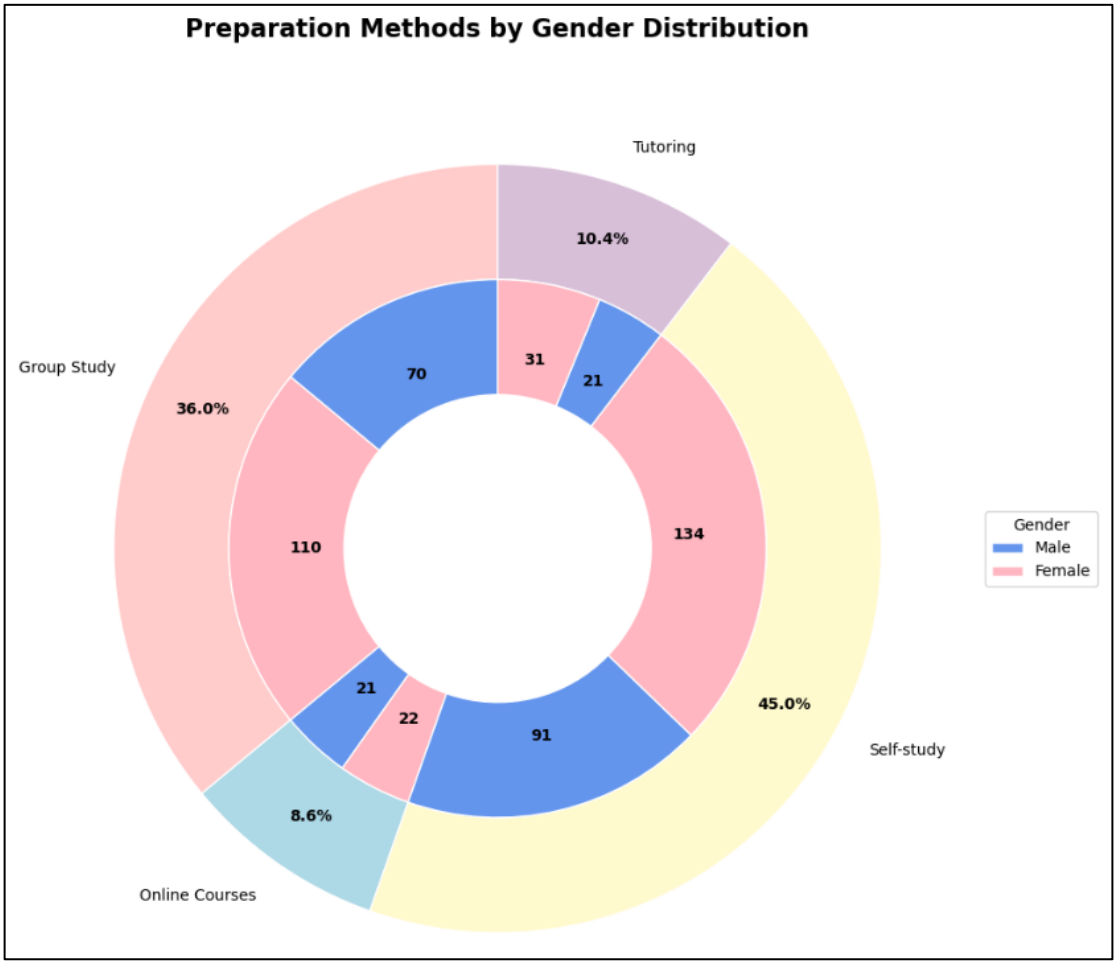


Figure 9.9 Ring Chart: Preparation Methods by Gender Distribution

```python
# Map preparation methods to English only
preparation_mapping = {
    'Self-study / Belajar sendiri': 'Self-study',
    'Group Study / Belajar berkumpulan': 'Group Study',
    'Tutoring / Tuisyen': 'Tutoring',
    'Online Courses / Kursus dalam talian': 'Online Courses'
}
data['c_how_preparation'] = data['c_how_preparation'].replace(preparation_mapping)

# Grouping data by preparation methods and gender
preparation_gender = data.groupby(['c_how_preparation', 'c_gender']).size().unstack(fill_value=0)

# Flatten data for the chart
outer_labels = preparation_gender.index
outer_values = preparation_gender.sum(axis=1)
inner_values = preparation_gender.values.flatten()

# Calculate percentages for outer labels
outer_percentages = [f"{value / outer_values.sum() * 100:.1f}%" for value in outer_values]

# Colors for the chart
outer_colors = ['#FFCCCB', '#ADD8E6', '#FFFACD', '#D8BFD8']  # Colors for the outer ring (methods)
inner_colors = ['#6495ED', '#FFB6C1'] * len(preparation_gender.index)  # Colors for the inner ring (gender)

# Creating the chart
fig, ax = plt.subplots(figsize=(10, 10))
outer_ring, _ = ax.pie(
    outer_values, labels=outer_labels, radius=1, wedgeprops=dict(width=0.3, edgecolor='w'),
    startangle=90, colors=outer_colors
)
inner_ring, _ = ax.pie(
    inner_values, radius=0.7, wedgeprops=dict(width=0.3, edgecolor='w'),
    startangle=90, colors=inner_colors
)

# Adding percentages inside the outer ring
for i, wedge in enumerate(outer_ring):
    angle = (wedge.theta2 + wedge.theta1) / 2
    x, y = 0.85 * math.cos(math.radians(angle)), 0.85 * math.sin(math.radians(angle))  # Position inside the ring
    ax.text(
        x, y, f"{outer_percentages[i]}",  # Only percentages inside
        fontsize=10, fontweight='bold', color='black', ha='center', va='center'
    )

# Adding numbers in the inner ring
for i, wedge in enumerate(inner_ring):
    angle = (wedge.theta2 + wedge.theta1) / 2
    x, y = 0.5 * math.cos(math.radians(angle)), 0.5 * math.sin(math.radians(angle))  # Position inside the inner ring
    ax.text(
        x, y, f"{inner_values[i]}", fontsize=10, fontweight='bold', color='black', ha='center', va='center'
    )

# Adding a legend with matching colors
ax.legend(
    handles=[Patch(facecolor='#6495ED', edgecolor='w', label='Male'),
             Patch(facecolor='#FFB6C1', edgecolor='w', label='Female')],
    title="Gender", loc="center left", bbox_to_anchor=(1, 0, 0.5, 1), fontsize=10
)

# Title
ax.set_title("Preparation Methods by Gender Distribution", fontsize=16, fontweight='bold', pad=20)

plt.tight_layout()
plt.show()
```

Figure 9.10 Code Implementation of Preparation Methods by Gender Distribution

The diagram shows that self-study is the most popular preparation method among both males and females, comprising 45.0% of the total. Group study follows as the second most adopted method, contributing to 36.0%, with males slightly leading in participation. Online courses and tutoring are less common, accounting for 8.6% and 10.4%, respectively, showing limited usage. The percentages inside the outer ring provide a clear understanding of the method's overall adoption rate, while the inner ring reveals gender-specific participation.

This visualization helps educators and curriculum designers tailor resources and support strategies based on student preferences. For instance, since self-study is highly favored, additional resources like study guides or online tools could be developed to aid independent learning. Furthermore, recognizing the limited usage of tutoring and online courses can guide efforts to make these options more appealing or accessible, potentially bridging gaps in preparation preferences and promoting equitable learning opportunities.

9.1.6 Mathematics Difficulty Level and Place of Studying

Figure 9.11 compares students' perceived difficulty levels in calculus with their chosen study environments. Each bar represents the number of students associating their study place with a specific difficulty level, allowing a detailed analysis of how the location impacts academic challenges. Figure 9.12 shows the code on how to implement this visualization.

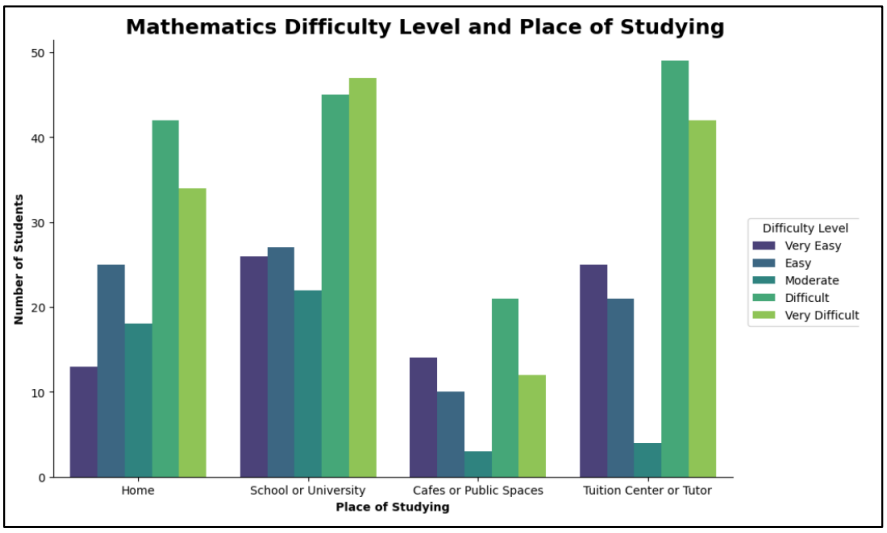


Figure 9.11 Clustered Bar Chart: Mathematics Difficulty Level and Place of Studying

```python
# Map difficulty levels to descriptive words
difficulty_mapping = {
    1.0: "Very Easy",
    2.0: "Easy",
    3.0: "Moderate",
    4.0: "Difficult",
    5.0: "Very Difficult"
}
data['c_difficulty_math'] = data['c_difficulty_math'].replace(difficulty_mapping)

# Map place of studying to English only
place_studying_mapping = {
    'At home / Di rumah (e.g., bedroom, study room)': 'Home',
    'At school or university / Di sekolah atau universiti (e.g., classroom, library)': 'School or University',
    'At cafes or other public spaces / Kafe atau tempat umum lain': 'Cafes or Public Spaces',
    'At a tuition center or with a private tutor / Di pusat tuisyen atau tutor persendirian': 'Tuition Center or Tutor'
}
data['c_place_studying'] = data['c_place_studying'].replace(place_studying_mapping)

# Set the order for difficulty levels
difficulty_order = ["Very Easy", "Easy", "Moderate", "Difficult", "Very Difficult"]

# Create the clustered bar chart
g = sns.catplot(
    data=data,
    x='c_place_studying',
    hue='c_difficulty_math',
    kind='count',
    palette='viridis',
    height=6,
    aspect=1.5,
    hue_order=difficulty_order  # Ensure the correct order of difficulty levels
)

# Title and labels
g.set_titles('')
g.set_axis_labels('Place of Studying', 'Number of Students', fontweight='bold')
g.set_xticklabels(rotation=0)
g.fig.suptitle('Mathematics Difficulty Level and Place of Studying', fontsize=18, fontweight='bold', y=1.02)

# Customize the legend
g._legend.set_title('Difficulty Level')
g._legend.set_bbox_to_anchor((1, 0.5))  # Move the legend outside the plot area
g._legend.set_frame_on(True)  # Add a border around the legend

# Show the plot
plt.show()
```

Figure 9.12 Code Implementation of Mathematics Difficulty Level and Place of Studying

This diagram reveals that students studying at tuition centers or with private tutors reported higher difficulty levels in calculus compared to those studying at home or in public spaces. Notably, students at school or university settings exhibited a more balanced distribution across all difficulty levels, suggesting that structured environments may provide adequate support for varying levels of complexity.

This analysis helps educators understand how different study environments influence students' academic experiences. By recognizing that certain settings like tuition centers are linked to perceived higher difficulty, targeted interventions can be implemented to provide additional support or resources, enhancing the learning outcomes in these environments.

9.1.7 Focus Levels in Mathematics and Their Learning Style

Figure 9.13 explores the relationship between students' learning styles and their self-reported focus levels during mathematics learning. Learning styles include Visual, Auditory, Kinesthetic, and Reading & Writing, while the focus levels are categorized as Very Low, Low, Moderate, and High. This analysis aims to identify patterns in how different learning styles impact the ability of students to concentrate effectively. Figure 9.14 shows the code on how to implement this visualization.

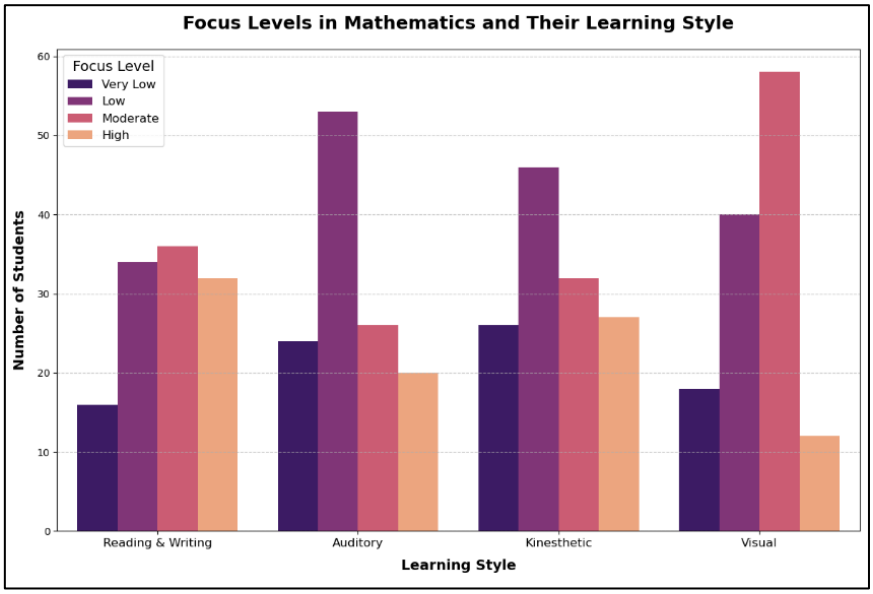


Figure 9.13 Clustered Bar Chart: Focus Levels in Mathematics and Their Learning Style

```python
# Mapping for learning style and focus level
learning_style_mapping = {
    "Visual (e.g., picture)": "Visual",
    "Auditory / Pendengaran (e.g., recording)": "Auditory",
    "Kinesthetic / Kinestetik (e.g., apply hands-on)": "Kinesthetic",
    "Reading & Writting / Membaca & Menulis": "Reading & Writing"
}

focus_level_mapping = {
    1.0: 'Very Low',
    2.0: 'Low',
    3.0: 'Moderate',
    4.0: 'High'
}

# Replace values with mappings
data['c_learning_style'] = data['c_learning_style'].replace(learning_style_mapping)
data['c_focus_math'] = data['c_focus_math'].replace(focus_level_mapping)

# Ensure all focus levels are represented, even if counts are zero
focus_order = ['Very Low', 'Low', 'Moderate', 'High']
data['c_focus_math'] = pd.Categorical(data['c_focus_math'], categories=focus_order, ordered=True)

# Plot the grouped bar chart
plt.figure(figsize=(12, 8))
sns.countplot(
    data=data,
    x='c_learning_style',
    hue='c_focus_math',
    palette='magma',
    hue_order=focus_order
)
plt.title('Focus Levels in Mathematics and Their Learning Style', fontsize=18, fontweight='bold', pad=20)
plt.xlabel('Learning Style', fontsize=14, fontweight='bold', labelpad=10)
plt.ylabel('Number of Students', fontsize=14, fontweight='bold', labelpad=10)

# Adjusting the legend and layout
plt.legend(
    title='Focus Level',
    fontsize=12,
    title_fontsize=14,
    loc='upper left'
)
plt.xticks(rotation=0, ha='center', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)  # Adding gridlines for better readability
plt.tight_layout()
plt.show()
```

Figure 9.14 Code Implementation of Focus Levels in Mathematics and Their Learning Style

The graph highlights interesting patterns in focus levels across different learning styles. Visual learners exhibit the highest number of students reporting "High" focus levels, whereas Auditory learners show a significant number in the "Moderate" focus category. Students who prefer Kinesthetic and Reading & Writing learning styles have more balanced distributions, with a fair representation in "Moderate" and "High" focus levels. Notably, the "Very Low" focus level is underrepresented, suggesting most students maintain at least a low level of focus regardless of learning style.

These findings are instrumental for educators aiming to tailor their teaching methods to individual learning styles. For instance, incorporating visual aids in teaching materials may help Visual learners achieve optimal focus. Similarly, for Auditory learners, strategies such as discussions or auditory instructions may prove beneficial. This insight

supports creating a more inclusive and effective learning environment by aligning educational approaches with students' natural learning preferences.

9.1.8 Focus Levels in Mathematics and Their Impact on Calculus Results

Figure 9.15 showcases the relationship between focus levels in mathematics and the corresponding calculus results (Pass/Fail). Focus levels are categorized into four distinct groups: Very Low, Low, Moderate, and High. Each bar in the chart represents the total number of students within a focus level, divided into segments indicating those who passed and failed the subject. This representation aids in identifying trends and patterns regarding how focus levels affect academic performance. Figure 9.16 shows the code on how to implement this visualization.

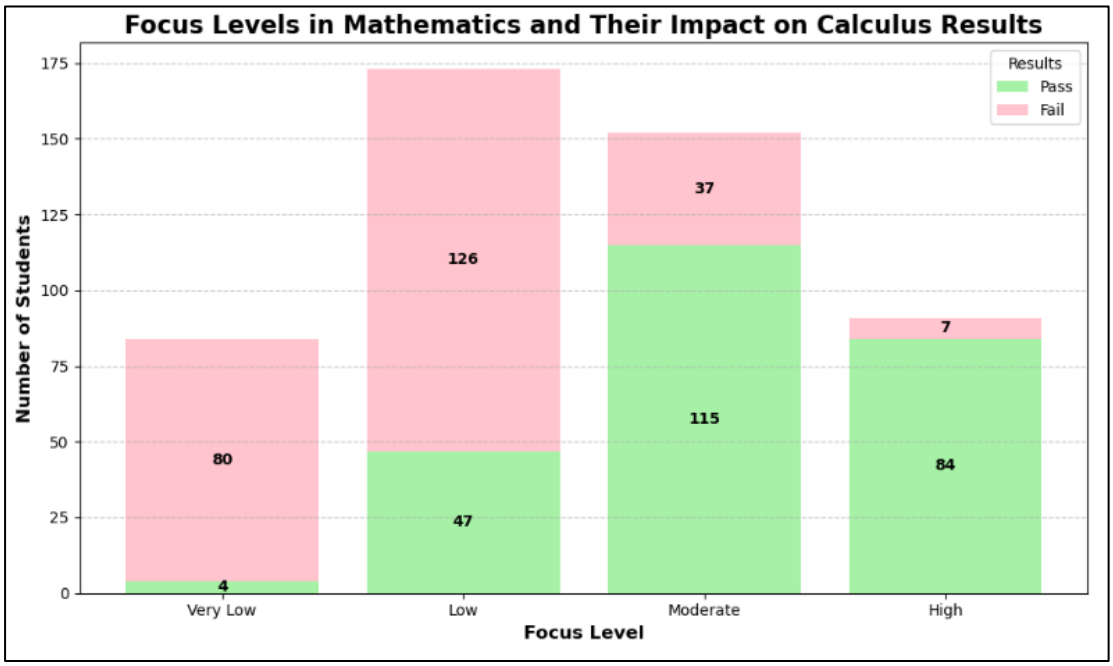


Figure 9.15 Stacked Bar Chart: Focus Levels in Mathematics and Their Impact on Calculus Results

```python
focus_level_mapping = {
    1.0: 'Very Low',
    2.0: 'Low',
    3.0: 'Moderate',
    4.0: 'High'
}

# Replace values in the 'c_focus_math' column using the mapping
data['Focus_Level'] = data['c_focus_math'].replace(focus_level_mapping)

# Grouping by Focus Level and Results
focus_results = data.groupby(['Focus_Level', 'c_result']).size().unstack(fill_value=0)

# Preparing data for visualization
focus_categories = focus_results.index
pass_values = focus_results['Pass / Lulus']
fail_values = focus_results['Fail / Gagal']

# Creating a stacked bar chart
fig, ax = plt.subplots(figsize=(10, 6))
bars_pass = ax.bar(focus_categories, pass_values, label='Pass', color='#90EE90', alpha=0.8)  # Light green for Pass
bars_fail = ax.bar(focus_categories, fail_values, bottom=pass_values, label='Fail', color='#FFB6C1', alpha=0.8)  # Light pink for Fail

# Adding labels to each bar
for bar_pass, bar_fail in zip(bars_pass, bars_fail):
    pass_height = bar_pass.get_height()
    fail_height = bar_fail.get_height()
    ax.text(bar_pass.get_x() + bar_pass.get_width() / 2, pass_height / 2, int(pass_height),
            ha='center', va='center', fontsize=10, color='black', weight='bold')
    ax.text(bar_fail.get_x() + bar_fail.get_width() / 2, pass_height + fail_height / 2, int(fail_height),
            ha='center', va='center', fontsize=10, color='black', weight='bold')

# Adding labels and title
ax.set_title("Focus Levels in Mathematics and Their Impact on Calculus Results", fontsize=16, fontweight='bold')
ax.set_xlabel("Focus Level", fontsize=12, fontweight='bold')
ax.set_ylabel("Number of Students", fontsize=12, fontweight='bold')
ax.legend(title="Results", fontsize=10)

# Adjusting layout for better spacing
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.grid(axis='y', linestyle='--', alpha=0.7)  # Adding gridlines for better readability
plt.tight_layout()

# Displaying the plot
plt.show()
```

Figure 9.16 Code Implementation of Focus Levels in Mathematics and Their Impact on Calculus Results

From the diagram, it is evident that students with higher focus levels, particularly those in the "High" and "Moderate" categories, have a significantly greater proportion of passing results compared to those in the "Low" or "Very Low" categories. For instance, the "High" focus level demonstrates a predominant number of pass outcomes, reflecting the positive impact of increased focus on academic achievement. Conversely, a substantial portion of students in the "Low" and "Very Low" focus levels fails the subject, underscoring the challenges faced by individuals with insufficient focus.

This analysis provides valuable insights for educators to prioritize interventions for students with lower focus levels. By identifying the focus-related challenges, targeted strategies such as personalized learning plans, counseling sessions, or motivational programs can be implemented to enhance students' engagement and performance. Ultimately, this information contributes to improving overall academic outcomes by addressing the root causes of failure.

9.2 Numerical Data Analysis

Numerical data consists of information represented in numeric form, which can be analyzed quantitatively using statistical methods and visualized effectively. These data are typically used to measure or quantify attributes and are often expressed as percentages, ratios, or absolute figures. For example, numerical data might include days of preparation, hours of study, hours spent on social media, or number of housemates.

In numerical data analysis, the focus is on identifying patterns, trends, and relationships within the data. This often involves summarizing the data through measures such as the mean, median, and standard deviation to provide insights into the central tendency and variability. Additionally, visualization tools such as histograms, line graphs, bar charts, and scatter plots are used to present the data in a way that makes trends and comparisons more accessible. For instance, percentages or ratios can be used to compare student success rates or visualize the correlation between study hours and academic performance.

By leveraging these techniques, numerical data analysis enables a deeper understanding of the dataset, supporting evidence-based decision-making and the identification of key factors that influence outcomes. It plays a vital role in interpreting results, validating hypotheses, and drawing actionable conclusions.

9.2.1 Study Hours and Preparation Days vs. Calculus Result

Figure 9.17 illustrates the relationship between the number of study hours, preparation days, and the calculus results (Pass/Fail). By categorizing the data into two distinct groups based on the results, this visualization provides a clearer understanding of the impact of study patterns on performance. The distribution of blue dots represents students who passed, while orange dots correspond to students who failed. Figure 9.18 shows the code on how to implement this visualization.

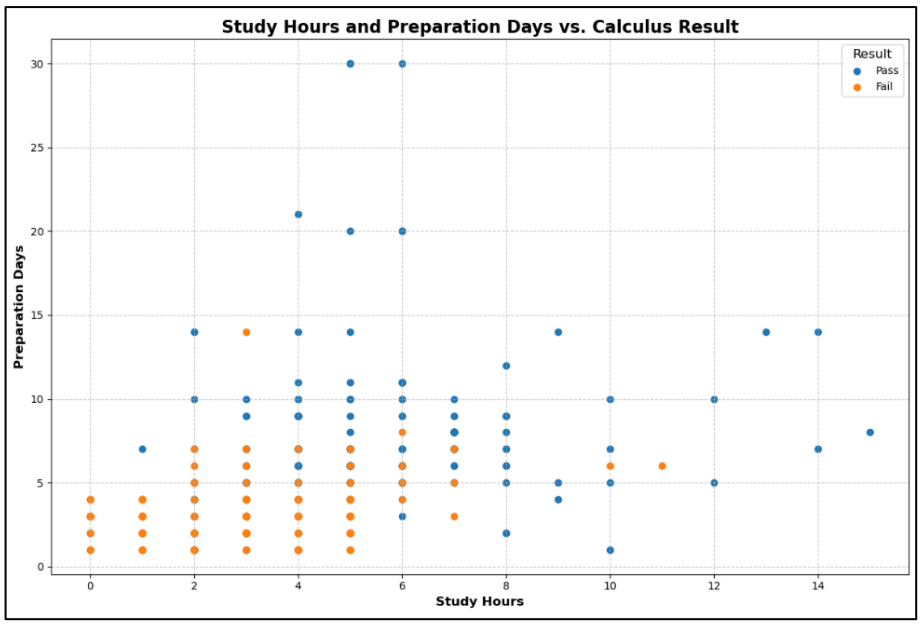


Figure 9.17 Scatter Plot: Study Hours and Preparation Days vs. Calculus Result

```
result_mapping = {
    'Pass / Lulus': 'Pass',
    'Fail / Gagal': 'Fail'
}
data['c_result'] = data['c_result'].replace(result_mapping)

# Create the scatter plot
plt.figure(figsize=(12, 8))
for result in data['c_result'].unique():
    subset = data[data['c_result'] == result]
    plt.scatter(subset['n_hours_studying'], subset['n_days_preparation'], label=result)

# Add title and labels
plt.title("Study Hours and Preparation Days vs. Calculus Result", fontsize=16, fontweight='bold')
plt.xlabel("Study Hours", fontsize=12, fontweight='bold')
plt.ylabel("Preparation Days", fontsize=12, fontweight='bold')
plt.legend(title="Result", fontsize=10, title_fontsize=12, loc='upper right')
plt.grid(True, linestyle='--', alpha=0.7)

# Show the plot
plt.tight_layout()
plt.show()
```

Figure 9.18 Code Implementation of Study Hours and Preparation Days vs. Calculus Result

From the visualization, we can observe that students who allocated more study hours and preparation days generally achieved better results, as indicated by the dense clustering of blue dots towards higher values of study hours and preparation days. Conversely, students with fewer preparation days and lower study hours tend to cluster within the failing category.

This scatter plot is a simple and effective tool to understand how preparation and study time affect results. It provides insights into areas for improvement, encouraging better time management and preparation strategies for future academic success.

9.2.2 The Relationship between Hours of Sleep on Calculus Result

Figure 9.19 illustrates the relationship between students' hours of sleep and their calculus results (Pass or Fail). Sleep is often regarded as a critical factor in cognitive function and academic success. By comparing the distribution of sleep hours across different result groups, this visualization aims to uncover potential patterns or trends that may explain the role of sleep in calculus result. The boxplot format is used to provide a detailed summary of data distribution, including key statistical measures like medians, quartiles, and outliers, offering a deeper understanding of the variability and consistency in students' sleep habits. Figure 9.20 shows the code on how to implement this visualization.

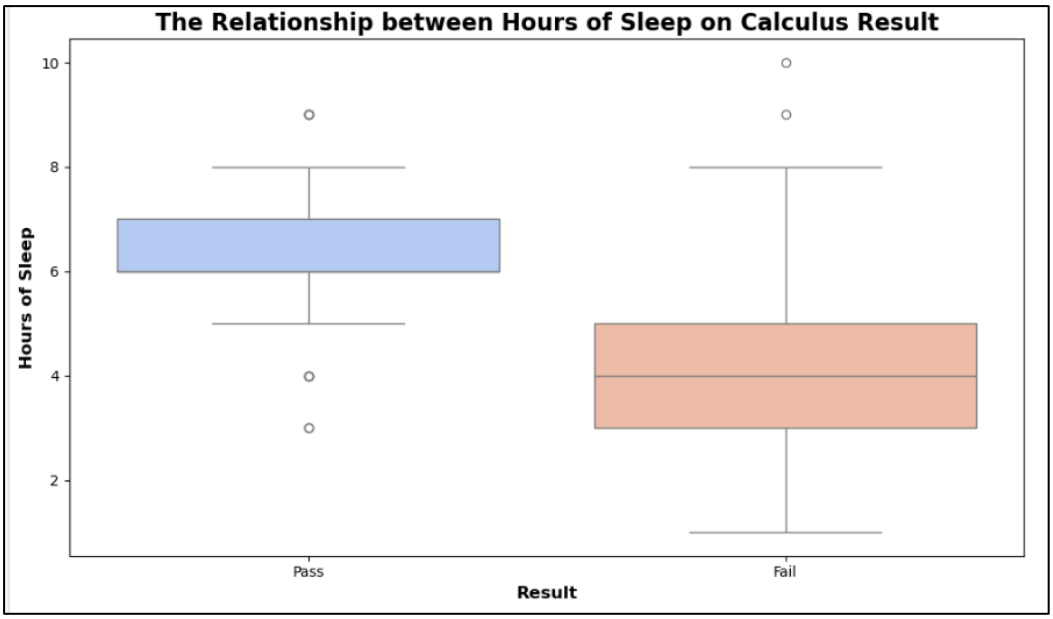


Figure 9.19 Boxplot: The Relationship between Hours of Sleep on Calculus Result

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data, x='c_result', y='n_hours_sleep', palette='coolwarm', hue=None)
plt.title("The Relationship between Hours of Sleep on Calculus Result", fontsize=16, fontweight='bold')
plt.xlabel("Result", fontsize=12, fontweight='bold')
plt.ylabel("Hours of Sleep", fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```

Figure 9.20 Code Implementation of The Relationship between Hours of Sleep on Calculus Result

The boxplot reveals a clear distinction in sleep patterns between students who passed and those who failed. Students who passed exhibit a relatively consistent range of sleep hours, with the median centered at six hours. Their interquartile range is narrower, suggesting greater uniformity in sleep habits among this group. Conversely, students who failed show a broader range of sleep hours, with outliers indicating extreme cases of either very short or excessive sleep durations. This variability in sleep patterns could point to less structured or erratic habits, which might negatively influence their academic performance.

This analysis highlights the importance of maintaining a consistent and moderate amount of sleep for improved academic performance. The findings can guide educators, parents, and students in emphasizing the need for structured sleep schedules as part of their daily routines. Moreover, it can serve as a starting point for further research or initiatives to promote healthier sleep habits among students, which could, in turn, enhance focus, memory retention, and overall performance in academic subjects such as calculus.

9.2.3 The Relationship Between Study Hours and Social Media Usage

The amount of time students allocate for studying and engaging with social media platforms can significantly influence academic outcomes. This relationship is pivotal for understanding the balance students maintain between their academic responsibilities and leisure activities. Figure 9.21 visualizes the correlation between the hours spent studying and hours spent on social media, categorized by their academic results (Pass or Fail). The inclusion of the Pearson correlation coefficient helps quantify the strength and direction of this relationship, providing a comprehensive understanding of this dynamic. Figure 9.22 shows the code on how to implement this visualization.

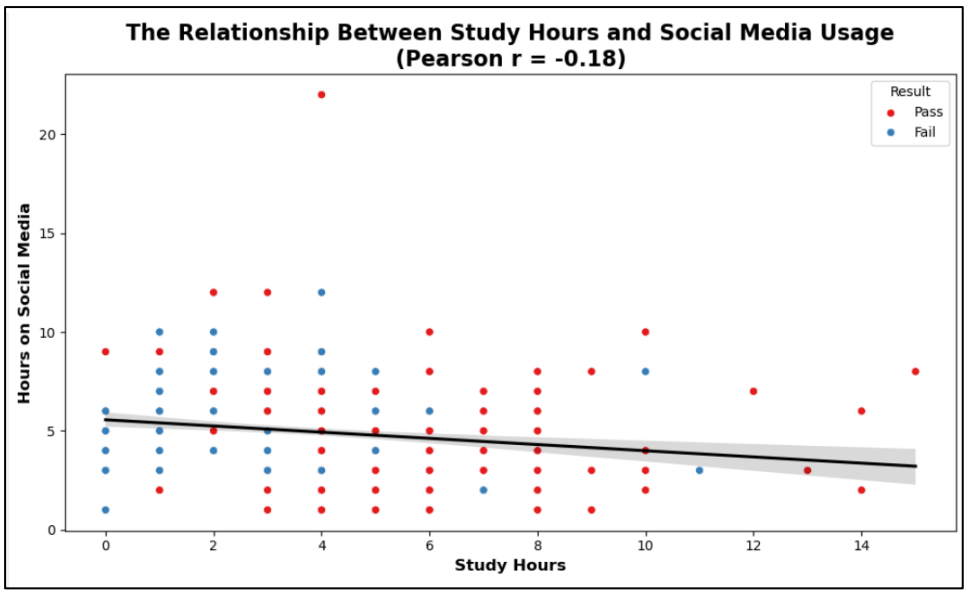


Figure 9.21 Scatter Plot with Regression Line: The Relationship Between Study Hours and Social Media Usage

```
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr

# Calculate the Pearson correlation coefficient and p-value
r_value, p_value = pearsonr(data['n_hours_studying'], data['n_hours_socmed'])

# Plot the scatterplot with the regression line
plt.figure(figsize=(10, 6))
sns.scatterplot(data=data, x='n_hours_studying', y='n_hours_socmed', hue='c_result', palette='Set1')
sns.regplot(data=data, x='n_hours_studying', y='n_hours_socmed', scatter=False, color='black')
plt.title(f"The Relationship Between Study Hours and Social Media Usage\n(Pearson r = {r_value:.2f})", fontsize=16, fontweight='bold')
plt.xlabel("Study Hours", fontsize=12, fontweight='bold')
plt.ylabel("Hours on Social Media", fontsize=12, fontweight='bold')
plt.legend(title="Result")
plt.tight_layout()
plt.show()
```

Figure 9.22 Code Implementation of The Relationship Between Study Hours and Social Media Usage

The scatter plot demonstrates a weak negative correlation (r = -0.18) between study hours and social media usage. This suggests that as students dedicate more time to studying, their social media usage tends to slightly decrease. However, the correlation is not strong, implying that social media usage is not heavily dependent on study hours. Students who passed (indicated in red) appear distributed across various levels of study and social media hours, whereas those who failed (indicated in blue) are also scattered with no distinct concentration. This distribution highlights the complexity of the relationship, where other factors may also play a role in determining academic performance.

Understanding the interplay between study habits and social media usage offers valuable insights for educators and parents. By acknowledging the weak negative correlation, strategies can be designed to encourage students to moderate social media engagement while optimizing their study schedules. Moreover, this analysis can guide further research into how other factors, such as motivation or time management skills, interact with these variables to influence academic success. This visualization can serve as a foundational tool to initiate discussions on achieving a balanced approach to academic and personal time management.

9.2.4 The Relationship between Preparation Days and Number of Additional Materials

The relationship between preparation days and the number of additional materials utilized is critical in understanding how students allocate their resources when preparing for exams. Figure 9.23 combined with a regression line, provides insight into whether a correlation exists between the time spent on preparation and the effort to gather supplementary materials. By exploring this relationship, we aim to determine if longer preparation periods encourage students to seek more learning aids, enhancing their study strategies and exam readiness. Figure 9.24 shows the code on how to implement this visualization.

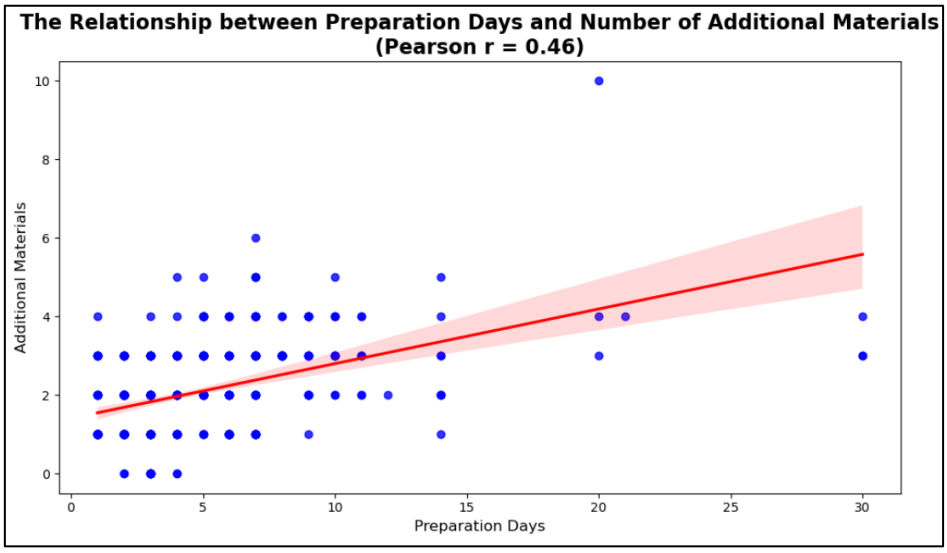


Figure 9.23 Scatter Plot with Regression Line: The Relationship between Preparation Days and Number of Additional Materials

```python
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import pearsonr

# Calculate Pearson correlation coefficient
x = data['n_days_preparation']
y = data['n_add_materials']
r, _ = pearsonr(x, y)

# Create the scatter plot with regression line
plt.figure(figsize=(10, 6))
sns.regplot(data=data, x='n_days_preparation', y='n_add_materials', scatter_kws={'color': 'blue'}, line_kws={'color': 'red'})

# Add title and labels
plt.title(f'The Relationship between Preparation Days and Number of Additional Materials\n(Pearson r = {r:.2f})', fontsize=16, fontweight='bold')
plt.xlabel('Preparation Days', fontsize=12)
plt.ylabel('Additional Materials', fontsize=12)

# Improve layout
plt.tight_layout()
plt.show()
```

Figure 9.24 Code Implementation of The Relationship between Preparation Days and Number of Additional Materials

The scatter plot demonstrates a positive correlation between the number of preparation days and the additional materials gathered, with a Pearson correlation coefficient of 0.46. This indicates a moderate relationship, suggesting that students who allocate more days for preparation are slightly more inclined to use additional resources. However, the variability in the data also highlights instances where students relied on a minimal number of resources regardless of extended preparation periods, reflecting individual study preferences.

Understanding this correlation helps educators and policymakers devise strategies to promote resourceful studying habits among students. By encouraging earlier preparation and providing access to quality supplementary materials, educators can potentially enhance academic performance. Additionally, insights from this analysis can guide the creation of tailored resources to meet the varying needs of students, ensuring more effective and efficient preparation techniques.

9.2.5 The Relationship between Hours of Exercising and Number of Meals

This visualization aims to explore the relationship between the number of hours students spend exercising and their meal frequency. By employing a 2D density plot with overlaid individual data points, Figure 9.25 provides a comprehensive view of the density and distribution of these two variables. Understanding these patterns is crucial for examining the correlation between students' physical activity and their dietary habits, which may impact their academic performance and well-being. Figure 9.26 shows the code on how to implement this visualization.

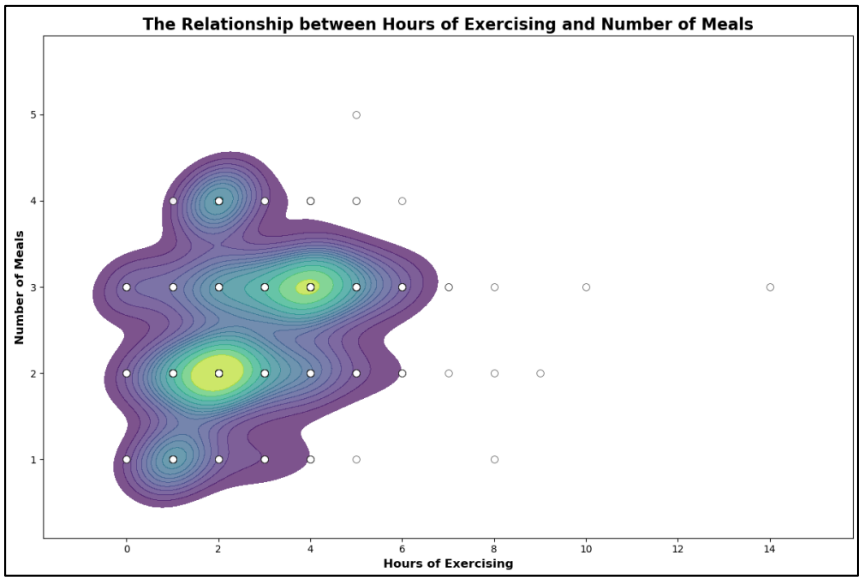


Figure 9.25 2D Density Plot: The Relationship between Hours of Exercising and Number of Meals

```
plt.figure(figsize=(12, 8))

sns.kdeplot(
    data=data,
    x='n_hours_exercising',
    y='n_meals',
    cmap='viridis',  # High-contrast color map
    fill=True,
    alpha=0.7,  # Transparency for overlapping
    levels=15,  # More levels for smoother gradients
    thresh=0.1  # Minimum density threshold
)

# Overlay scatter points for individual data visibility
sns.scatterplot(
    data=data,
    x='n_hours_exercising',
    y='n_meals',
    color='white',
    edgecolor='black',
    alpha=0.7,  # Transparency for better visibility
    s=50  # Size of the points
)

# Adding labels and title
plt.title('The Relationship between Hours of Exercising and Number of Meals', fontsize=16, fontweight='bold')
plt.xlabel('Hours of Exercising', fontsize=12, fontweight='bold')
plt.ylabel('Number of Meals', fontsize=12, fontweight='bold')

# Improve layout
plt.tight_layout()
plt.show()
```

Figure 9.26 Code Implementation of The Relationship between Hours of Exercising and Number of Meals

The analysis highlights distinct clusters in the density plot, indicating a concentration of students who engage in fewer hours of exercising while consuming one to three meals per day. There are also fewer instances of students exercising extensively while maintaining consistent meal patterns. This suggests a general trend where students with moderate to low exercise levels align their meal frequencies within a typical range.

This visualization is particularly useful for educators and counselors to identify lifestyle patterns among students and offer guidance on balancing physical activity and nutrition. For students, this analysis underscores the importance of integrating healthy habits, such as regular exercise and balanced meals, into their routines to enhance their academic performance and overall well-being.

9.2.6 The Relationship Between Hours of Sleep, Number of Devices, and Calculus Results

Figure 9.27 illustrates the relationship between the number of devices owned by students, their average hours of sleep, and their academic results in calculus (Pass or Fail). By categorizing data based on results, the line plot reveals distinct patterns for each group, providing insights into how device ownership and sleep habits might influence academic outcomes. The inclusion of confidence intervals further enhances the understanding of data variability within each category, making the trends more interpretable and meaningful. Figure 9.28 shows the code on how to implement this visualization.

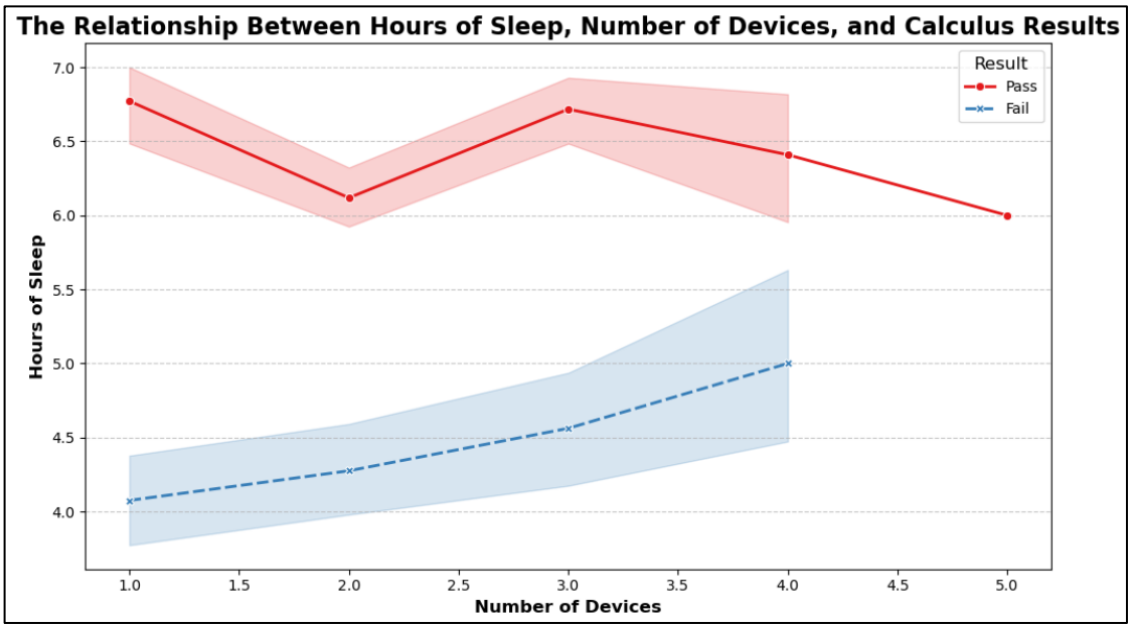


Figure 9.27 Line Plot with Confidence Interval: The Relationship Between Hours of Sleep, Number of Devices, and Calculus Results

```python
# Line plot with c_result as hue
plt.figure(figsize=(10, 6))
sns.lineplot(
    data=data,
    x='n_devices',
    y='n_hours_sleep',
    hue='c_result',
    style='c_result',  # Different line styles for Pass/Fail
    markers=True,      # Add markers to the lines
    palette='Set1',    # Color palette for distinct categories
    linewidth=2
)

# Adding title and labels
plt.title('The Relationship Between Hours of Sleep, Number of Devices, and Calculus Results', fontsize=16, fontweight='bold')
plt.xlabel('Number of Devices', fontsize=12, fontweight='bold')
plt.ylabel('Hours of Sleep', fontsize=12, fontweight='bold')

# Adjusting legend
plt.legend(title='Result', fontsize=10, title_fontsize=12, loc='upper right')

# Adding gridlines for better readability
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Enhancing layout
plt.tight_layout()

# Show the plot
plt.show()
```

Figure 9.28 Code Implementation of The Relationship Between Hours of Sleep, Number of Devices, and Calculus Results

The line plot highlights contrasting behaviors between students who passed and those who failed. For the pass group, sleep hours peaked with fewer devices but declined steadily as the number of devices increased, potentially suggesting that higher device ownership leads to distractions or reduced sleep quality. On the other hand, students in the failure group showed a consistent increase in sleep hours with more devices, possibly indicating a lack of structured routines or excessive use of devices for non-educational purposes. These findings emphasize the role of balanced device usage and sufficient sleep in academic success.

This visualization provides actionable insights for educators and students. Educators can use these findings to create awareness programs on managing device usage and promoting better sleep hygiene. Students can benefit from reflecting on their habits and adopting healthier routines, such as reducing screen time before bedtime and maintaining a consistent study-sleep balance. Such interventions can contribute to improved academic performance and overall well-being.

9.2.7 Distribution of Number of Housemates by Calculus Result

The living environment can significantly influence a student's academic performance, particularly in a shared living arrangement. The number of housemates may affect a student's ability to concentrate, study, and ultimately succeed in their studies. To explore this relationship, Figure 9.29 was created to visualize the distribution of the number of housemates for students who passed and those who failed. This diagram allows us to examine if there are notable differences in living arrangements based on academic outcomes. Figure 9.30 shows the code on how to implement this visualization.

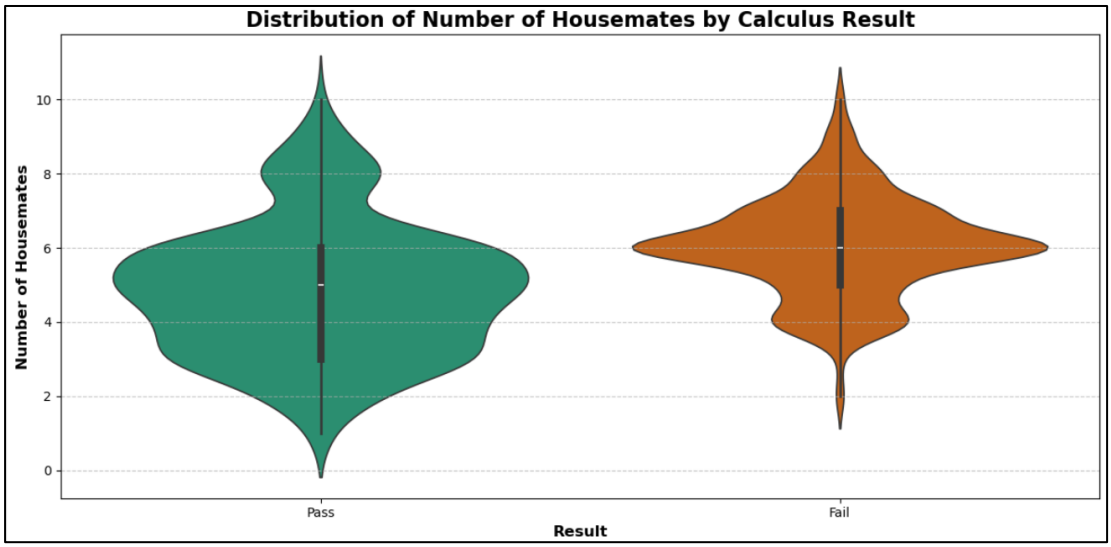


Figure 9.29 Violin Plot: Distribution of Number of Housemates by Calculus Result

```
plt.figure(figsize=(12, 6))
sns.violinplot(
    data=housemates_data,
    x='c_result',
    y='n_housemate',
    palette='Dark2'
)
plt.title('Distribution of Number of Housemates by Calculus Result', fontsize=16, fontweight='bold')
plt.xlabel('Result', fontsize=12, fontweight='bold')
plt.ylabel('Number of Housemates', fontsize=12, fontweight='bold')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Figure 9.30 Implementation of Distribution of Number of Housemates by Calculus Result

The violin plot highlights that the distribution of housemates/dormmates differs between students who passed and those who failed. For students who passed, the distribution appears to be more concentrated, with most having around four to six housemates/dormmates. Conversely, students who failed show a broader range of housemate numbers, indicating more variability in their living arrangements. The medians for both groups indicate that successful students tend to have a slightly more balanced number of housemates compared to those who failed.

This visualization provides educators and policymakers with valuable insights into how living conditions might influence academic performance. By understanding the relationship between housemate numbers and results, institutions can consider offering better support to students in shared accommodations, such as creating study-friendly environments or promoting more balanced living arrangements to improve academic success.

9.2.8 Comparison of Average Study Habits (Pass vs. Fail)

Figure 9.31 provides an in-depth comparison of the average study habits of students who pass versus those who fail. It examines multiple key factors, including study hours, preparation days, sleep hours, social media hours, and exercise hours. This visualization aims to offer insights into how these behaviors differ between successful and less successful academic outcomes. Figure 9.32 shows the code on how to implement this visualization.

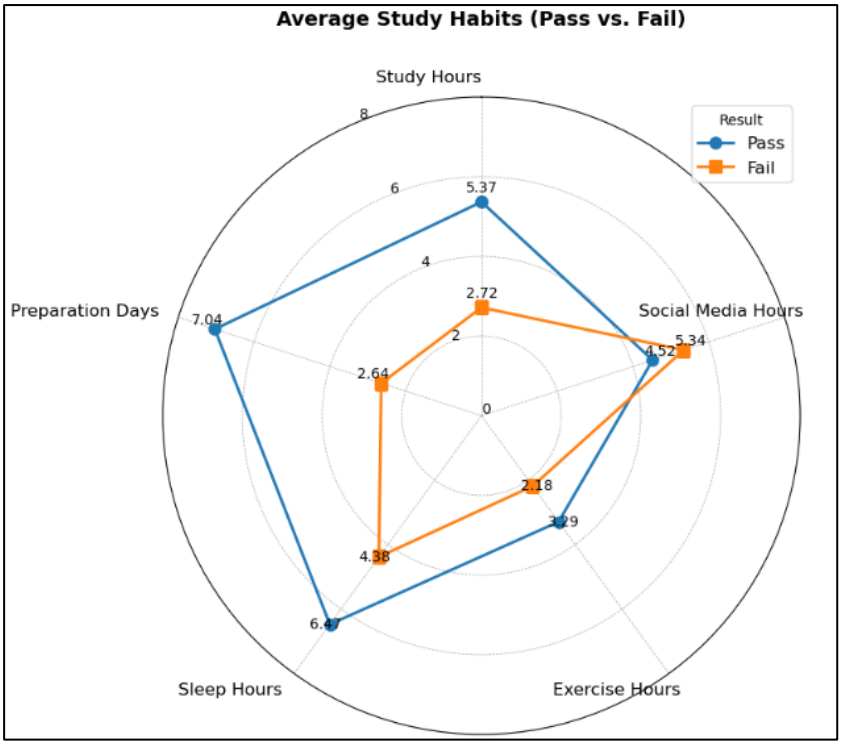


Figure 9.31 Radar Chart: Average Study Habits (Pass vs. Fail)

```python
categories = {
    'n_hours_studying': 'Study Hours',
    'n_days_preparation': 'Preparation Days',
    'n_hours_sleep': 'Sleep Hours',
    'n_hours_exercising': 'Exercise Hours',
    'n_hours_socmed': 'Social Media Hours'
}

# Calculate means using .loc for label-based indexing
success_means = data_final.loc[data_final['c_result'] == 'Pass / Lulus', categories.keys()].mean()
fail_means = data_final.loc[data_final['c_result'] == 'Fail / Gagal', categories.keys()].mean()

# Convert to numpy arrays *after* using .loc
success_means_np = success_means.values
fail_means_np = fail_means.values

N = len(categories)
angles = np.linspace(0, 2 * np.pi, N, endpoint=False).tolist()

# Close the polygon (important for fill)
success_means_np = np.concatenate((success_means_np, [success_means_np[0]]))
fail_means_np = np.concatenate((fail_means_np, [fail_means_np[0]]))
angles += angles[:1]

fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(polar=True))

# Adjusted grid with minor ticks
ax.set_rgrids(np.arange(min(min(success_means_np), min(fail_means_np)) - 0.5,
                        max(max(success_means_np), max(fail_means_np)) + 0.5, step=1))
ax.set_theta_zero_location('N')  # Set north as the starting point

# Thicker lines and colorblind-friendly colors
ax.plot(angles, success_means_np, marker='o', markersize=8, linestyle='-', color='tab:blue', label="Pass", linewidth=2)
ax.plot(angles, fail_means_np, marker='s', markersize=8, linestyle='-', color='tab:orange', label="Fail", linewidth=2)

# Rotated category labels with descriptive names
category_labels = [categories[cat] for cat in categories.keys()]
ax.set_xticks(angles[:-1])  # Set ticks before labels (KEY CHANGE)
ax.set_xticklabels(category_labels, rotation=45, ha='right', fontsize=12)

# Data labels with rounding and position adjustment
for angle, value in zip(angles[:-1], success_means_np[:-1]):
    plt.text(angle, value + 0.2, round(value, 2), ha='center', va='bottom', fontsize=10)

for angle, value in zip(angles[:-1], fail_means_np[:-1]):
    plt.text(angle, value + 0.2, round(value, 2), ha='center', va='bottom', fontsize=10)

ax.set_yticks(np.arange(0, max(max(success_means_np), max(fail_means_np)) + 1, step=2))  # Set y-ticks
ax.tick_params(axis='y', which='both', labelsize=10)  # Adjust both major and minor tick labels
plt.legend(title="Result", fontsize=12)
plt.title("Radar Chart: Average Study Habits (Pass vs. Fail)", fontsize=14, fontweight='bold', y=1.1)  # Adjusted title position
ax.grid(color='gray', linestyle='--', linewidth=0.5, alpha=0.7)  # Grid styling

# Calculate and print insights based on sleep data
average_sleep_diff = round(success_means_np[2] - fail_means_np[2], 2)
sleep_label = "Pass" if success_means_np[2] > fail_means_np[2] else "Fail"

print(f"On average, students who {sleep_label} get {average_sleep_diff} more hours of sleep per night.")
```

```
On average, students who Pass get 2.09 more hours of sleep per night.
```

Figure 9.32 Code Implementation of Average Study Habits (Pass vs. Fail)

The chart reveals a distinct contrast between the two groups. Notably, on average, students who pass get 2.09 more hours of sleep per night than those who fail, suggesting that sufficient rest plays a vital role in academic success. Furthermore, students who pass dedicate significantly more time to preparation and study while limiting their social media hours compared to their peers who fail.

This analysis provides actionable insights for educators and students alike. For educators, the chart highlights the importance of promoting healthy habits, such as encouraging consistent preparation and adequate sleep, to enhance academic outcomes. For students, it serves as a reminder of the potential impact of lifestyle choices, such as managing distractions and prioritizing sleep, on their overall performance and well-being.

9.2.9 Preparation Days and Distance from Home by Calculus Result

Figure 9.33 provides a detailed comparison of two critical factors: preparation days and the distance from home, segmented by student outcomes (Pass or Fail). Preparation days represent the time spent actively studying and preparing for assessments, while distance from home reflects the geographical challenges students face. By examining these two metrics against the results, this visualization aims to uncover any correlation or trend that could influence academic performance. Figure 9.34 shows the code on how to implement this visualization.

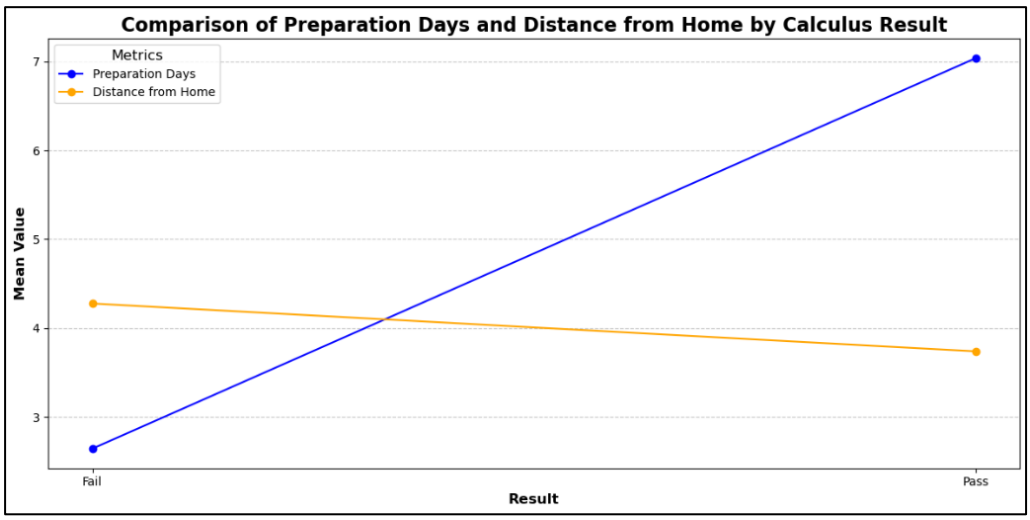


Figure 9.33 Line Graph: Preparation Days and Distance from Home by Calculus Result

```python
# Grouping data by `c_result` and calculating the mean
grouped_data = data.groupby('c_result')[['n_days_preparation', 'n_distance']].mean().reset_index()

# Line graph for `n_days_preparation` and `n_distance`
plt.figure(figsize=(12, 6))
plt.plot(grouped_data['c_result'], grouped_data['n_days_preparation'], marker='o', label='Preparation Days', color='blue')
plt.plot(grouped_data['c_result'], grouped_data['n_distance'], marker='o', label='Distance from Home', color='orange')

# Title and labels
plt.title('Comparison of Preparation Days and Distance from Home by Calculus Result', fontsize=16, fontweight='bold')
plt.xlabel('Result', fontsize=12, fontweight='bold')
plt.ylabel('Mean Value', fontsize=12, fontweight='bold')
plt.legend(title='Metrics', fontsize=10, title_fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Figure 9.34 Code Implementation of Preparation Days and Distance from Home by Calculus Result

The analysis reveals a notable trend where students who pass dedicate more preparation days compared to those who fail, as evidenced by the sharp incline in the blue line. On the contrary, the distance from home shows an inverse relationship, with students who pass typically residing closer to campus or their educational institution. These patterns suggest that consistent preparation plays a pivotal role in academic success, whereas distance from home may have a smaller but still noteworthy impact.

This visualization is instrumental for educators and policymakers to design targeted interventions. Understanding the importance of preparation days can prompt the creation of structured study schedules or programs to help students better manage their time. Similarly, acknowledging the challenges posed by long commutes could lead to enhanced access to online resources or support for students traveling from afar, ensuring equitable learning opportunities.

10.0 Result and Discussion

The implemented machine learning model in this project has demonstrated exceptional performance, achieving an impressive testing accuracy of 98%. This underscores the effectiveness of the K-Nearest Neighbors (KNN) algorithm in predicting pass or fail outcomes in Calculus based on various input features. The model's reliability is further validated through K-Fold cross-validation experiments, ensuring its robustness across diverse subsets of the dataset. With this high level of accuracy, the model is well-equipped to identify patterns and relationships within the data, providing a solid foundation for predicting and understanding student performance in Calculus.

11.0 Conclusion

In the challenging domain of Calculus education, students showcase varying strategies influenced by their individual learning approaches. Successful students often emphasize consistent practice, effective time management, and utilizing available resources to excel in the subject. Conversely, students who struggle may focus on basic understanding and cautious effort to avoid overwhelming themselves. Both groups recognize the importance of personalized learning strategies but approach the balance between effort and comprehension differently. High-achieving students may seek collaborative learning environments, while struggling students may prioritize self-paced study. The real-world factors affecting Calculus performance involve diverse learning methods, teaching dynamics, and personal circumstances, making any generalized assessment complex and nuanced.

12.0 Recommendations/limitations

i) Recommendations

The project demonstrates significant potential but also has areas for improvement. One of the key recommendations is to enhance data collection by including additional factors that may influence student performance, such as attendance records, motivation levels, access to resources, and participation in extracurricular activities. Expanding the dataset size and diversity would also improve the model's generalizability and robustness. Incorporating feature engineering techniques, such as adding prior mathematics scores or study group participation as predictors, can further strengthen the model's performance. Additionally, experimenting with other machine learning algorithms like Support Vector Machines or Ensemble Models and applying hyperparameter tuning could enhance predictive accuracy. The system could also provide actionable feedback for at-risk students, such as recommending study strategies, connecting them with tutors, or scheduling extra classes. To ensure the tool's effectiveness, the model should be validated with real-world outcomes and integrated into existing Learning Management Systems (LMS) for seamless adoption by educators and students.

ii) Limitations

Despite its promise, the project has several limitations. The quality and quantity of data used are critical constraints; a limited or imbalanced dataset can result in biased predictions, and missing data can hinder the model's accuracy. The current feature set may not account for all relevant factors, particularly non-academic influences like psychological well-being or socio-economic status, which are harder to quantify. Additionally, there is a risk of overfitting if the model is too complex, potentially reducing its performance on unseen data. The generalizability of the model may also be restricted to specific contexts or demographic groups, limiting its applicability to other educational environments. Furthermore, the interpretability of machine learning predictions, especially from complex models, remains a challenge, making it difficult for educators to understand the reasoning behind predictions. Lastly, the model's predictions require extensive validation against real-world outcomes to confirm their reliability and effectiveness. Addressing these limitations and implementing the recommendations will enhance the system's practicality, accuracy, and overall impact.

REFERENCES

Alnuaimi, A. F. A. H., & Albaldawi, T. H. K. (2024). An overview of machine learning classification techniques. *BIO Web of Conferences*, *97*. https://doi.org/10.1051/bioconf/20249700133

Aloyod, N. B. (2024). CORRELATES OF STUDENTS' PERFORMANCE IN CALCULUS. *Psych Educ*, *19*(3), 2024–1759. https://doi.org/10.5281/zenodo.11049918

Ayebo, A., Ukkelberg, S., & Assuah, C. (2017). Success in Introductory Calculus: The Role of High School and Pre-calculus Preparation. *International Journal of Research in Education and Science (IJRES)*, *3*(1), 11–19. http://www.ijres.net/

Hidayati, N., & Hermawan, A. (2021). K-Nearest Neighbor (K-NN) algorithm with Euclidean and Manhattan in classification of student graduation. *Journal of Engineering and Applied Technology*, *2*(2). https://doi.org/10.21831/jeatech.v2i2.42777

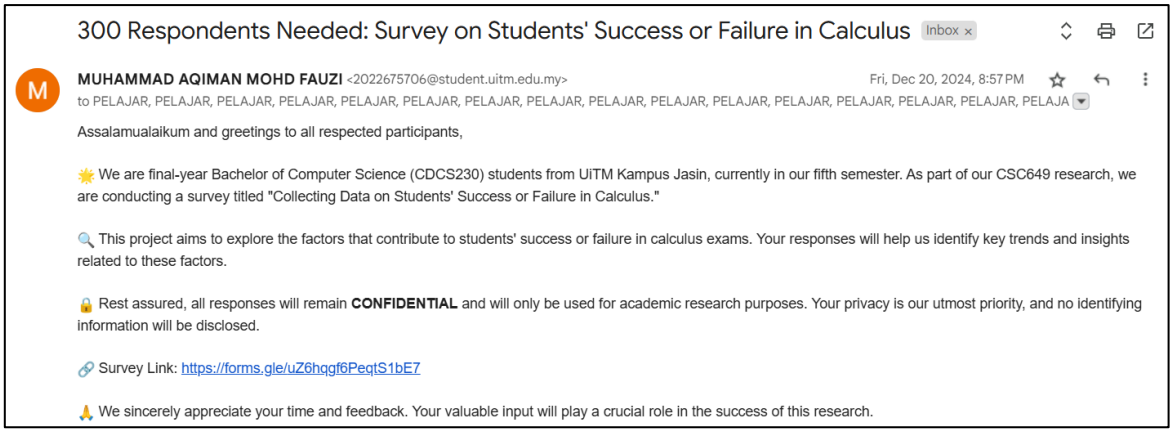
Jawthari, M., & Stoffová, V. (2021). Predicting students' academic performance using a modified kNN algorithm. *Pollack Periodica*, *16*(3), 20–26. https://doi.org/10.1556/606.2021.00374

Yağcı, M. (2022). Educational data mining: prediction of students' academic performance using machine learning algorithms. *Smart Learning Environments*, *9*(1). https://doi.org/10.1186/s40561-022-00192-z

APPENDICES

https://forms.gle/Rmvq7d26c1PiqVrr8

https://tinyurl.com/mvuep53z



300 Respondents Needed: Survey on Students' Success or Failure in Calculus Inbox ×

MUHAMMAD AQIMAN MOHD FAUZI <2022675706@student.uitm.edu.my> Fri, Dec 20, 2024, 8:57 PM

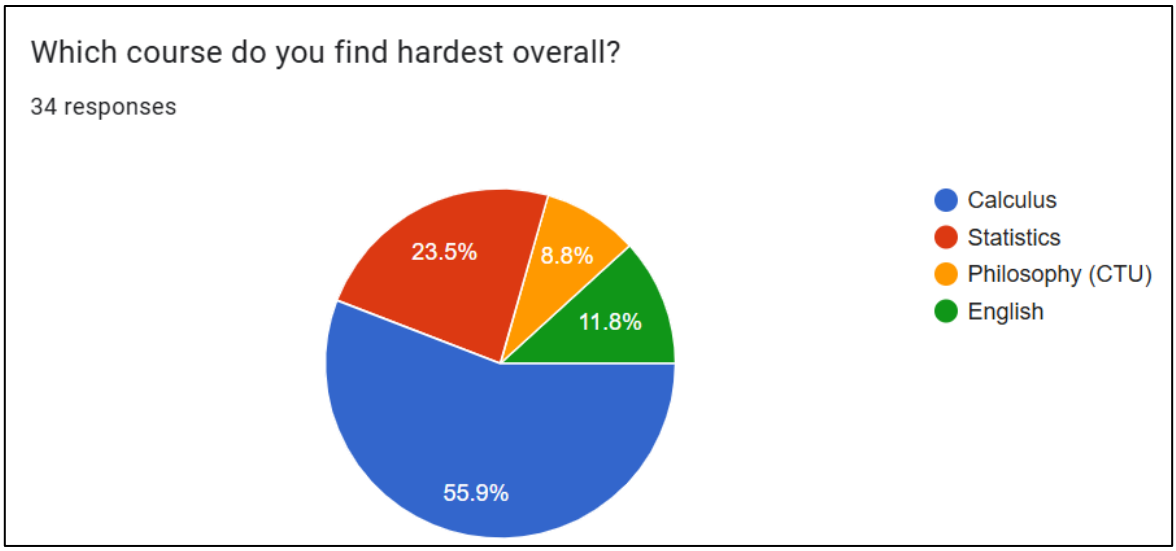
to PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJAR, PELAJA

Assalamualaikum and greetings to all respected participants,

🌟 We are final-year Bachelor of Computer Science (CDCS230) students from UiTM Kampus Jasin, currently in our fifth semester. As part of our CSC649 research, we are conducting a survey titled "Collecting Data on Students' Success or Failure in Calculus."

🔍 This project aims to explore the factors that contribute to students' success or failure in calculus exams. Your responses will help us identify key trends and insights related to these factors.

🔒 Rest assured, all responses will remain **CONFIDENTIAL** and will only be used for academic research purposes. Your privacy is our utmost priority, and no identifying information will be disclosed.

🔗 Survey Link: https://forms.gle/uZ6hqgf6PeqtS1bE7

🙏 We sincerely appreciate your time and feedback. Your valuable input will play a crucial role in the success of this research.